

CM 2900- Industry Based AI Software Project Interim Report

PredictiX : AI-Powered Predictive Maintenance & Smart Ticket Categorization for Asset Management

Team NeuroMinds - AI Group 02

Name	Index
Ekanayake D.B	235514B
Abeywickrama W.S.J.	235502L
Senevirathna P.T.B	235536T
Wijesinghe W.M.A.N	235547D

Supervised by:

Ms. K.A.D.T. Kulawansa
Ms. Uvini Matara Arachchi

Client:

Zone 24x7, Sri Lanka
460 Nawala Rd, Sri Jayawardenepura Kotte 10107

**BSc (Hons) Artificial Intelligence
Department of Computational Mathematics
Faculty of Information Technology University of Moratuwa 2025**

Declaration

We declare that this report is our own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published or unpublished work of others has been acknowledged in the text, and a list of references is given.

Names of Students

Ekanayake D.B

Abeywickrama W.S.J

Senevirathna P.T.B

Wijesinghe W.M.A.N

Signatures of Students



Date: 12.02.2026

Supervised by:

Names of Supervisors

Mrs. K.A.D.T. Kulawansa

Ms. Uvini Matara Arachchi

Signatures of Supervisors



Date

12/02/2026

12/02/2026

Abstract

This interim report depicts the creation of PredictiX : AI Powered Predictive Maintenance & Ticket Classification for Asset Management, an application designed to predict warehouse assets' maintenance. It improves operational efficiency, reduces unexpected equipment failures, and supports data-driven decision-making. Traditional maintenance approaches rely on reactive or scheduled methods, which often lead to increased downtime and unnecessary maintenance costs. PredictiX addresses these limitations by combining asset lifecycle management, intelligent ticket handling, and advanced predictive analytics into a unified platform.

The system maintains records, asset logs, and real-time operational data to predict equipment failures before they occur. Machine learning models analyze asset health trends and failure probabilities, while natural language processing techniques automatically categorize maintenance tickets and generate meaningful insights. A Retrieval-Augmented Generation (RAG) approach is used to produce explainable asset and warehouse-level reports, translating complex analytics into clear, actionable summaries for users.

PredictiX supports role-based access for administrators and warehouse staff, offering interactive dashboards, automated report generation, and an AI-powered chatbot assistant. By integrating predictive intelligence with human-in-the-loop control, the platform ensures accuracy, transparency, and trust in maintenance decisions. Overall, PredictiX provides a scalable, cost-effective solution for modern warehouse management, enabling proactive maintenance strategies, reduced operational risks, and improved asset reliability.

Table of Contents

Chapter 1: Introduction.....	8
1.1 Introduction.....	8
1.2 Problem in Brief.....	8
1.3 Aim and Objectives.....	9
1.3.1 Aim.....	9
1.3.2 Objectives.....	9
1.4 Proposed Solution.....	10
1.5 Structure of the Report.....	11
1.6 Summary.....	11
Chapter 2: Literature Review.....	13
2.1 Introduction.....	13
2.2 Related Systems.....	13
2.2.1 Asset Panda.....	13
2.2.2 ManageEngine AssetExplorer.....	14
2.2.3 Flexera One.....	15
2.2.4 Reftab.....	15
2.2.5 Wasp AssetCloud Complete.....	16
2.2.6 Lansweeper.....	16
2.2.7 IBM Maximo Application Suite.....	17
2.2.8 InvGate Asset Management.....	17
2.3 Feature Comparison table with PredictiX.....	18
2.4 Summary.....	19
Chapter 3: Proposed Solution.....	20
3.1 Introduction.....	20
3.2 Frontend technologies.....	20
3.2.1 NextJS.....	20
3.2.2 Tailwind CSS.....	21
3.2.3 shadcn/ui.....	21
3.3 Backend technologies.....	21
3.3.1 Render.....	21
3.3.2 FastAPI.....	22
3.4 Database Technologies.....	22
3.4.1 Supabase.....	22
3.5 AI Technologies.....	23
3.5.1 Predictive Maintenance and Failure Prediction (XGBoost).....	23
3.5.2 Maintenance Cost Estimation (XGBoost).....	23
3.5.3 Prediction Explainability (SHAP).....	24

3.5.4 Maintenance Ticket Categorization and Priority Prediction (DistilBERT).....	24
3.5.5 Maintenance Ticket Summarization (DistilBART).....	24
3.5.6 Semantic Retrieval and Knowledge Grounding (Sentence Transformers and RAG).....	24
3.5.7 Report Generation and Validation (RAG and LLM with Multi Step Agents).....	24
3.5.9 AI-Powered Chatbot and Multi-Agent Interaction (RAG with LLM).....	25
3.6 Software Development Process Model.....	26
3.7 User Role, Input- Process - Output of the System.....	27
3.7.1 User Role.....	27
3.7.2 Inputs - Process - Output.....	28
Inputs.....	28
Processes.....	28
Outputs.....	28
3.8 Processes.....	28
3.9 Our Approach.....	29
Chapter 4: Analysis and Design.....	30
4.1 Introduction.....	30
4.2 Analysis.....	30
4.2.1 Functional Requirements.....	30
4.2.1.1 User Registration and Authentication.....	30
4.2.1.2 Asset Creation and Management.....	31
4.2.1.3 Predictive Maintenance and Cost Estimation.....	31
4.2.1.4 Maintenance Ticket Management.....	31
4.2.1.5 AI-Powered Chatbot Interaction.....	31
4.2.1.6 Reporting and Visualization.....	32
4.2.1.7 Explainability and Transparency.....	32
4.2.2 Non-Functional Requirements.....	32
4.3 Design.....	33
4.3.1 Use case Diagram.....	34
4.3.2 Activity Diagrams.....	35
4.3.2.1 Login.....	35
4.3.2.2 Asset Management (Create, View, Update, Delete Assets).....	36
4.3.2.3 Search Assets.....	37
4.3.2.4 Asset Report Generation.....	38
4.3.2.5 Create Ticket.....	39
4.3.2.6 View, Update, Delete Tickets.....	40
4.3.2.8 Search Tickets.....	42
4.3.2.9 View Profile.....	43
4.3.2.10 Manage Users (Create, View, Update, Delete Users).....	44
4.3.2.11 Search Users.....	45
4.3.2.12 View warehouse insights.....	46
4.3.2.13 Warehouse Report Generation.....	47

4.3.2.14 AI Chatbot.....	48
4.3.4 Sequence Diagrams.....	49
4.3.4.1 Login UI.....	49
4.3.4.2 Asset Management (Create, View, Update, Delete Assets).....	50
4.3.4.3 Search Assets.....	51
4.3.4.4 Asset Report Generation.....	52
4.3.4.5 Create Ticket.....	53
4.3.4.6 View, Update, Delete tickets.....	54
4.3.4.7 Review Ticket.....	55
4.3.4.8 View Warehouse Insights.....	55
4.3.4.9 Warehouse Report Generation.....	56
4.3.4.10 Profile Profile.....	57
4.3.4.11 User Management (Create, View, Update, Delete User Accounts).....	58
4.3.4.12 Search Users.....	59
4.3.3.13 AI Chatbot.....	60
4.3.5 ER Diagram.....	61
4.3.6 Class Diagram.....	62
4.3.7 Component Diagram.....	63
Chapter 5: Implementation.....	64
5.1 Introduction.....	64
5.2 Implementation Process.....	64
5.3 Summary.....	65
Chapter 6: References.....	66
Chapter 7: Appendix- Individual Contributions.....	67

List of Figures/ Tables

Figure 4.3.1: Use Case Diagram

Figure 4.3.2.1: Login Activity Diagram

Figure 4.3.2.2: Asset Management Activity Diagram

Figure 4.3.2.3: Search Assets Activity Diagram

Figure 4.3.2.4 Asset Report Generation Activity Diagram

Figure 4.3.2.5 Create Ticket Activity Diagram

Figure 4.3.2.6 View, Update, Delete Ticket Diagram

Figure 4.3.2.7 Review Tickets Activity Diagram

Figure 4.3.2.8 Search Tickets Activity Diagram

Figure 4.3.2.9 View Profile Activity Diagram

Figure 4.3.2.10 Manage Users (Create, View, Update, Delete Users) Activity Diagram

Figure 4.3.2.11 Search Users Activity Diagram

Figure 4.3.2.12 View warehouse insights Activity Diagram

Figure 4.3.2.13 Warehouse Report Generation Activity Diagram

Figure 4.3.2.14 AI Chatbot Activity Diagram

Figure 4.3.3.1 login UI Sequence Diagram

Figure 4.3.3.2 Asset Management (Create, View, Update, Delete Assets) Sequence Diagram

Figure 4.3.3.3 Search Assets Sequence Diagram

Figure 4.3.3.4 Asset Report Generation Sequence Diagram

Figure 4.3.3. 5: Create Ticket Sequence Diagram

Figure 4.3.3. 6: View, Update, Delete Tickets Sequence Diagram

Figure 4.3.3.8: Review Ticket Sequence Diagram

Figure 4.3.3. 9: View Warehouse Insights Sequence Diagram

Figure 4.3.3.10 Warehouse Report Generation Sequence Diagram

Figure 4.3.3.11 Profile Profile Sequence Diagram

Figure 4.3.3.12 User Management (Create, View, Update, Delete User Accounts) Sequence Diagram

Figure 4.3.3.13 Search Users Sequence Diagram

Figure 4.3.3.14 AI Chatbot Sequence Diagram

Figure 4.3.4 ER Diagram.

Figure 4.3.5 Class Diagram

Figure 4.3.6 Component Diagram

List of Abbreviations

Abbreviation	Meaning
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
ML	Machine Learning
API	Application Programming Interface
DB	Database
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language
UI	User Interface
NLP	Natural Language Processing
SRS	Software requirements specification
UML	Unified Modeling Language
SHAP	SHapely Additive exPlanations
XAI	Explainable Artificial Intelligence

Chapter 1: Introduction

1.1 Introduction

With the advancement of automation and digital transformation in industrial and warehouse environments, efficient maintenance of equipment has become a critical requirement. Warehouses rely on machinery such as conveyor belts, forklifts, automated storage systems, and sorting units to ensure smooth operations. Any unexpected failure of such equipment can lead to operational downtime, increased maintenance costs, and safety risks.

Recent advancements in Artificial Intelligence and Machine Learning have enabled organizations to move from traditional maintenance approaches toward intelligent, data-driven maintenance systems. This chapter introduces the motivation, objectives, and overall direction of the proposed system, PredictiX, which aims to combine predictive maintenance with intelligent ticket classification.

1.2 Problem in Brief

Warehouse and industrial environments depend heavily on continuous operation of equipment such as conveyor belts, forklifts, and automated handling systems. Equipment failure in such environments leads to unplanned downtime, operational delays, increased maintenance costs, and potential safety risks. Despite the availability of large volumes of maintenance records and sensor data, most organizations still rely on reactive or schedule-based maintenance strategies that do not utilize this data effectively.

In addition, maintenance issue reporting is commonly handled through manually created maintenance tickets written in free-text form. These tickets must be read and categorized by human operators, which is time-consuming, inconsistent, and prone to errors. Incorrect categorization can delay issue resolution and affect maintenance prioritization.

The core problem is the absence of an integrated intelligent system that can:

- Predict equipment failures before they occur using historical maintenance and sensor data
- Automatically categorize maintenance tickets using AI-based text analysis
- Estimate the potential cost of maintenance activities to support cost-aware decision-making
- Provide timely, data-driven insights to support maintenance decision-making

As a result of these limitations, organizations experience increased downtime, inefficient use of maintenance resources, delayed response to critical issues, and avoidable maintenance costs. This project addresses these challenges by proposing an AI-driven solution that integrates predictive maintenance, maintenance cost estimation, and intelligent maintenance ticket management within a unified decision-support platform.

1.3 Aim and Objectives

1.3.1 Aim

The main aim of the PredictiX system is to design and develop an AI-powered warehouse asset management and predictive maintenance platform that enables early detection of equipment failures, improves maintenance planning, estimates potential maintenance costs and supports informed decision-making through intelligent analytics, explainable models, and automated insights.

1.3.2 Objectives

1. **To design a role-based warehouse asset management system**

Provide separate access levels for admins and users to ensure secure and controlled interaction with assets, tickets, reports, and insights.

2. **To implement asset management**

Enable creation, viewing, updating, searching, and deletion of warehouse assets along with maintenance schedules, operational status, and historical logs.

3. **To develop predictive maintenance capabilities for warehouse assets**

Analyze asset logs and sensor readings to calculate health scores, predict failure probabilities, estimate maintenance dates, and display prediction confidence levels.

4. **To estimate maintenance costs for informed decision-making**

Develop AI-based maintenance cost estimation capabilities to predict potential repair or service costs based on asset characteristics, failure risk, historical maintenance patterns, and operational impact.

5. **To visualize asset health and failures effectively**

Present charts such as health score trends, failure probability forecasts, and maintenance timelines to support informed maintenance planning.

6. **To automate maintenance ticket creation and management**

Allow users and administrators to create, update, review, and track maintenance tickets with AI-assisted priority prediction, categorization, and status monitoring.

7. **To provide intelligent ticket summarization**

Generate ticket summaries, maintain update histories, and document resolution details to improve transparency and accountability.

8. **Generate AI-driven asset and warehouse insight reports**

Produce individual asset reports and overall warehouse reports highlighting health distribution, risk levels, ticket trends, and maintenance workload.

9. **To support warehouse-level decision-making through dashboards**

Display key performance indicators such as critical assets, assets requiring immediate attention, estimated maintenance costs, active tickets, and recent maintenance activities within a centralized dashboard.

10. **Integrate an AI-powered chatbot for quick information access**

Enable users to explore asset details, maintenance schedules, generate reports, predictions, and ticket information using natural language interaction.

11. **Ensure system usability, scalability, and accessibility**

Provide responsive user interfaces, theme customization, and efficient backend processing suitable for real-world warehouse environments.

12. **To provide explainable and transparent AI-driven predictions**

Apply explainable AI techniques to interpret predictive maintenance and cost estimation outputs by identifying key factors influencing asset health scores, failure probabilities, and maintenance cost predictions.

13. **To validate and ground AI-generated content**

Ensure that AI-generated reports, insights, and chatbot responses are grounded in verified system data, predictive outputs, and historical records to prevent misinformation and hallucinations.

1.4 Proposed Solution

The proposed solution, PredictiX, is an intelligent, AI-driven maintenance management system composed of multiple integrated and modular components. The system aggregates historical maintenance records, available sensor readings, asset metadata, and free-text maintenance ticket descriptions to enable data-driven maintenance planning and decision support.

Machine learning models are used to predict equipment failure probabilities, estimate asset health conditions, and forecast potential maintenance costs. Natural language processing (NLP) techniques are used to automatically categorize, prioritize, and summarize incoming maintenance tickets. In addition, explainable AI methods are applied to enhance transparency by identifying key factors influencing predictive outcomes.

The PredictiX system consists of the following core modules:

- **Asset Management Module** for maintaining comprehensive records of warehouse assets, including operational status, maintenance history, and scheduling information
- **Predictive Maintenance Module** for calculating asset health scores, predicting failure risks, estimating maintenance timelines, and forecasting potential maintenance costs
- **Maintenance Ticket Management Module** for intelligent ticket creation, automated categorization, priority prediction, summarization, and lifecycle tracking
- **Reporting and Decision Support Module** for generating AI-driven asset-level and warehouse-level insight reports grounded in historical data and predictive outputs
- **AI-Powered Chatbot Module** for natural language interaction, enabling users to query assets, tickets, predictions, cost estimates, and reports efficiently

The modular architecture of PredictiX ensures scalability, maintainability, and flexibility, allowing individual components to be extended or enhanced independently. By integrating predictive analytics, cost estimation, NLP-based ticket intelligence, and explainable AI within a unified platform, PredictiX provides a comprehensive solution for proactive maintenance management in warehouse environments.

1.5 Structure of the Report

Font Type - Times New Roman
Chapter Headings Font size - 16

Font Type - Times New Roman
Headings Font size - 14

Sub heading font size - 12 Bold
Content Font size - 12

1.6 Summary

The objectives of the PredictiX system focus on delivering an intelligent, role-based warehouse asset management and predictive maintenance solution that integrates natural language processing (NLP), retrieval-augmented generation (RAG), tabular machine learning, explainable AI (XAI), and multi-agent system concepts. By combining comprehensive asset lifecycle management with AI-driven failure prediction, maintenance cost estimation, and automated ticket handling, the system

aims to reduce unexpected equipment downtime and improve maintenance cost efficiency.

Interactive dashboards, detailed visualizations, and insight-driven reports are used to support informed decision-making at both the individual asset level and the overall warehouse level. In addition, secure user management, natural language chatbot assistance, and a modular, scalable system design ensure usability, transparency, and adaptability to real-world warehouse environments. Collectively, these objectives guide the development of a proactive, data-driven maintenance platform that enhances operational reliability, supports effective resource allocation, and enables informed maintenance planning.

Chapter 2: Literature Review

2.1 Introduction

Effective asset management is a critical component of modern organizations, enabling efficient tracking, maintenance, and utilization of physical and digital resources. Warehouse and industrial organizations increasingly depend on digital systems to manage equipment, monitor asset usage, and coordinate maintenance activities. Modern asset management platforms typically provide features such as asset registers, lifecycle tracking, scheduled maintenance reminders, barcode or QR-based identification, reporting dashboards, and ticket-based maintenance workflows. While some enterprise-level solutions extend these capabilities through IoT integration and predictive analytics, many mid-range platforms remain largely focused on record keeping and process management rather than proactive failure prevention and decision support.

This chapter reviews eight widely used asset management and maintenance-related systems relevant to the scope of the PredictiX project. Each system is examined in terms of its intended purpose, core functionalities, and practical application in real operational environments. Particular attention is given to the challenges and limitations observed across these platforms, including gaps in predictive maintenance capabilities, limited support for maintenance cost awareness, insufficient use of AI for ticket handling, and usability or scalability concerns in analytics and reporting.

The objective of this review is to critically analyze existing asset management solutions, identify their strengths and limitations, and highlight opportunities for improvement through the adoption of advanced technologies such as AI-driven predictive maintenance, intelligent ticket automation, and data-driven analytical insights. These findings provide a clear idea for defining the requirements and design direction of the proposed PredictiX application.

2.2 Related Systems

This section reviews the capabilities of existing asset management solutions, their limitations, and proposes potential enhancements. The analysis is structured based on key features commonly found in leading asset management applications, Asset Panda, Reftab, Wasp AssetCloud, AssetExplorer, Flexera One, InvGate, and IBM Maximo.

2.2.1 Asset Panda

Asset Panda is a cloud-based asset tracking and management platform used for managing asset lifecycle data across a wide range of industries. It supports barcode and QR code tracking, asset assignment, inspection checklists, maintenance notifications, and repair tracking. It also provides mobile access, record level attachments (manuals, photos, warranties), configurable fields, role-based views, and reporting exports. Asset Panda's ecosystem includes integrations with identity providers and enterprise tooling, and it markets AI-assisted features such as auto-fill and column mapping for easier data onboarding.

Challenges and Limitations

- **Performance and stability issues:** Users frequently report slow loading, syncing problems, and lag especially when many users operate simultaneously. This can reduce adoption in fast moving environments.
- **UI/UX friction and learning curve:** The interface is often described as clunky or overwhelming for new users, which increases training time and data-entry mistakes.
- **Reporting and audit pain points:** Reporting setup and auditing workflows are commonly described as difficult, suggesting the system may not fully support clear operational visibility for all organizations.
- **Limited predictive intelligence:** While Asset Panda supports reminders and tracking, its core function remains operational logging rather than predictive maintenance forecasting.

2.2.2 ManageEngine AssetExplorer

ManageEngine AssetExplorer is an IT Asset Management (ITAM) platform designed to provide centralized tracking of IT and non-IT assets with a strong emphasis on discovery and inventory management. It includes hardware/software inventory discovery, a CMDB for relationship mapping, software license compliance tracking, procurement workflows, contract renewal alerts, and reporting tools. It is commonly positioned for organizations managing network-connected IT assets (Windows/Linux/macOS) and supports mobile access.

Challenges and Limitations

- **Cost and licensing restrictions:** Users report that licensing can become expensive and that key capabilities may be locked behind higher-tier plans, which can limit adoption for smaller teams.
- **Complex setup and customization effort:** Initial configuration is widely described as time-consuming and overwhelming, making onboarding difficult without dedicated expertise.
- **Scalability and ROI concerns:** Some users report limited scalability in multi-customer contexts where additional instances are required, impacting long-term return on investment. (ROI).
- **Limited physical predictive maintenance focus:** Although strong in IT Asset Management (ITAM), it does not naturally provide predictive failure forecasting for mechanical warehouse equipment without additional external analytics.

2.2.3 Flexera One

Flexera One is a cloud-based enterprise IT management solution focused on software asset management, license optimization, cloud cost control, and compliance. The platform is built for complex enterprise environments and includes automation for license reconciliation, audit readiness, spend forecasting, and large scale IT visibility. While it supports hardware asset tracking as part of its broader suite, its primary value remains software/license and cloud financial optimization rather than physical equipment maintenance.

Challenges and Limitations

- **Complex implementation:** Organizations often require extensive time and expertise to integrate multiple sources and fully configure normalization and reporting workflows.
- **Steep learning curve:** It can overwhelm new users and may require formal training and professional services to use effectively.
- **Cost barriers:** The pricing structure is typically suitable for large enterprises, which can make it unrealistic for smaller organizations or academic deployments.
- **Predictive maintenance gap for physical assets:** Flexera One is not purpose-built for condition-based monitoring or predictive failure estimation for industrial warehouse equipment.

2.2.4 Reftab

Reftab is a lightweight, cloud-based asset management platform designed for simplicity and quick adoption. It supports asset tracking across categories, check-in/check-out workflows, barcode scanning, mobile-first operations, offline mode, maintenance scheduling reminders, reporting exports, and role-based access control. It is commonly used by small to mid-sized organizations that need a practical replacement for spreadsheet-based tracking with minimal overhead.

Challenges and Limitations

- **Manual data maintenance:** Users note that some asset details do not sync cleanly from integrations, creating ongoing manual cleanup and reduced time savings.
- **Reporting depth:** Reporting is useful for basic scenarios but can fall short for complex analytics and may push users toward Excel or BI tools for deeper analysis.
- **Offline sync reliability:** While offline mode is a strength, sync failures and conflict handling are still reported, which can risk data consistency.

- **Limited predictive capability:** Reftab focuses on tracking and reminders and does not provide predictive maintenance forecasting or failure probability modelling.

2.2.5 Wasp AssetCloud Complete

Wasp AssetCloud Complete is a physical asset tracking platform oriented around barcode/RFID-based identification and lifecycle management. It supports check-in/check-out, location tracking across branches, maintenance scheduling, depreciation calculation, audit trails, and basic reporting. This tool fits organizations that want structured physical inventory tracking with compliance-friendly logs of asset changes.

Challenges and Limitations

- **RFID cost and operational dependency:** RFID hardware cost can be a real barrier, and the system still depends on staff discipline to check assets in/out correctly for accurate records.
- **Manual maintenance updates:** Maintenance scheduling exists, but it relies heavily on users manually updating changes, which reduces reliability in busy environments.
- **Limited reporting customization:** Report customization is noted as restricted, making it harder to produce tailored decision-support views.
- **Audit log searchability:** Audit trails can become large and difficult to search effectively over time without stronger indexing and retrieval.

2.2.6 Lansweeper

Lansweeper is an IT asset discovery and network inventory tool focused on automatically scanning networks to identify devices, installed software, and vulnerability-related issues. It provides detailed asset discovery, dashboards, reports, and compliance-related insights, making it valuable for cybersecurity and IT operations. Its strengths are strongest in network-connected assets rather than warehouse mechanical equipment.

Challenges and Limitations

- **Duplicate detection and scan performance:** Large network scans can be slow, and duplicate device identification can occur, requiring cleanup and tuning.
- **Advanced reporting requires expertise:** Custom report creation may require SQL knowledge, which becomes a barrier for non-technical operational teams.
- **Reliance on regular sync/updates:** Vulnerability insights and asset accuracy depend on frequent scanning and synchronization.

- **Not designed for mechanical predictive maintenance:** The platform does not address physical failure prediction for warehouse machinery as a core feature.

2.2.7 IBM Maximo Application Suite

IBM Maximo Application Suite is a well-known enterprise asset management solution that combines asset registry, work order management, IoT monitoring, and AI-assisted maintenance features. It provides capabilities such as asset health scoring dashboards, real-time sensor integration, and ML-based predictive functionality through Maximo modules, supporting end-to-end maintenance flows from monitoring to work order generation.

Challenges and Limitations

- **High enterprise cost:** Maximo licensing and deployment costs are typically beyond what small organizations or academic projects can support.
- **Heavy infrastructure and setup:** Implementation complexity and learning curve are significant, requiring skilled teams and long onboarding cycles.
- **Limited flexibility for experimentation:** As a vendor-controlled enterprise system, it can be difficult to freely test custom ML/NLP pipelines or quickly modify modules for research purposes.

2.2.8 InvGate Asset Management

InvGate Asset Management is an IT asset and service management platform that includes an asset inventory layer alongside helpdesk-style ticketing through related service desk workflows. It supports incident logging, categorization, and structured workflows that can be useful when studying ticket classification and prioritization. However, it is less aligned with physical predictive maintenance and IoT-based monitoring, as its design is primarily IT service focused.

Challenges and Limitations

- **Limited relevance to physical equipment maintenance:** The tool is more suitable for IT-related tickets and does not naturally support sensor-driven industrial equipment workflows.
- **Lack of predictive maintenance functionality:** It does not include built-in predictive failure forecasting or asset health prediction modules for warehouse machinery.
- **IoT and sensor integration not central:** Without sensor integration support, it cannot provide condition-based monitoring for industrial assets as required in warehouse environments.

2.3 Feature Comparison table with PredictiX

Feature	PredictiX	Asset Panda	Manage Engine	Flexera One	Reftab	Wasp Asset Cloud	Lans weeper	IBM Maximo	Inv Gate
Role-Based Access Control	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Asset Lifecycle Tracking	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Maintenance Scheduling	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes
Predictive Maintenance (Failure Prediction)	Yes	No	No	No	No	No	No	Yes	No
Maintenance Cost Estimation (AI based)	Yes	No	No	Partial	No	No	No	Partial	No
Asset Health Scoring	Yes	No	No	No	No	No	No	Yes	No
Explainable AI for Predictions (SHAP)	Yes	No	No	No	No	No	No	No	No
AI-Based Ticket Categorization	Yes	No	No	No	No	No	No	No	No
AI-Based Ticket Priority Prediction	Yes	No	No	No	No	No	No	No	No

Feature	PredictiX	Asset Panda	Manage Engine	Flexera One	Reftab	Wasp Asset Cloud	Lans weeper	IBM Maximo	Inv Gate
AI-Based Ticket Summarization	Yes	No	No	No	No	No	No	No	No
Retrieval-Augmented Report Generation (RAG)	Yes	No	No	No	No	No	No	No	No
Warehouse Level AI Insights	Yes	No	No	No	No	No	No	Partial	No
AI-Powered Chatbot	Yes	No	No	No	No	No	No	No	No
Prediction Confidence Indicators	Yes	No	No	No	No	No	No	No	No

2.4 Summary

Existing asset management systems provide a strong foundation for managing assets across organizations, including features for tracking, maintenance, reporting, and user management. However, common limitations include:

- Lack of predictive maintenance and AI-assisted decision-making.
- Performance degradation under large-scale operations.
- Manual processes in asset tracking and work order management.
- Complex reporting and navigation interfaces.

These gaps support the need for a unified solution like PredictiX that combines asset lifecycle management with predictive maintenance, cost aware decision support, and AI-assisted ticket intelligence, while also grounding outputs through retrieval based evidence (RAG) and improving transparency using explainable AI (XAI).

Chapter 3: Proposed Solution

3.1 Introduction

Modern warehouse operations depend heavily on the continuous availability and reliability of physical assets such as conveyors, forklifts, sensors, and automated systems. Unexpected equipment failures can lead to operational delays, increased maintenance costs, and safety risks. Traditional reactive maintenance approaches are no longer sufficient in data-driven environments.

PredictiX introduces an AI-powered Warehouse Asset Management and Predictive Maintenance System designed to monitor asset health, predict potential failures and costs with explanations, and streamline maintenance workflows. The system integrates asset tracking, predictive analytics, maintenance ticketing, and warehouse-level insights into a unified platform. By combining real-time data visualization with intelligent predictions with explanations, the solution supports proactive decision-making for both administrators and operational users.

The proposed system focuses on improving asset reliability, reducing downtime and maintenance cost, and enhancing maintenance efficiency through automation and data-driven insights. Role-based access, interactive dashboards, and an integrated chatbot further enhance usability, making the platform suitable for real-world warehouse management scenarios.

3.2 Frontend technologies

The frontend of the PredictiX system is built using modern web technologies to deliver a responsive and interactive user interface. It supports dynamic data visualization for presenting asset health trends, failure probability forecasts, ticket distributions, and warehouse-level analytics.

The frontend also includes essential usability features such as search and filtering, dark and light theme support, and real-time data updates, enabling efficient interaction with system data and analytics.

3.2.1 NextJS

Next.js is a modern React-based framework used to build fast and responsive web applications. It supports features such as server-side rendering, client-side routing, and optimized performance, making it suitable for data-intensive dashboards and interactive user interfaces.

In the PredictiX application, Next.js is used to develop the frontend application that allows users to interact with assets, maintenance tickets, dashboards, and AI-driven insights. Its ability to efficiently manage dynamic content enables smooth visualization of predictive maintenance results, reports, and real-time updates. By combining performance, flexibility, and a strong developer ecosystem, Next.js provides a reliable frontend foundation for the PredictiX platform.

3.2.2 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that enables rapid and consistent user interface styling through predefined utility classes. It allows developers to design responsive layouts and custom visual styles without relying on large, predefined CSS files.

In PredictiX, Tailwind CSS is used to style the frontend application, ensuring responsive design across different screen sizes and devices. Its flexibility supports quick iteration of dashboard layouts and visual components, contributing to a clean, maintainable, and scalable user interface.

3.2.3 shadcn/ui

shadcn/ui is a modern component library built on top of React that provides reusable, accessible, and customizable UI components. Instead of abstracting components behind a rigid framework, it allows developers to directly integrate and modify components within the application codebase, offering greater flexibility and control over the user interface.

In the PredictiX system, shadcn/ui is used to build consistent and user-friendly interface elements such as forms, tables, dialogs, and dashboards. Its clean design and accessibility-focused components help ensure a professional and intuitive user experience across asset management, ticket handling, and analytics views.

3.3 Backend technologies

The backend of the PredictiX is responsible for managing core application logic, data access, and communication between system components. It handles user authentication and role-based access control, ensuring that users interact with assets, tickets, and reports according to their assigned permissions. The backend also manages API communication between the frontend interface, the database, and integrated AI services.

In addition, the backend oversees key operational functions such as asset record management, maintenance ticket workflows, report generation, user administration, and system notifications. It exposes secure APIs that allow the frontend to retrieve predictive maintenance results, maintenance cost estimates, and analytical insights generated by the AI models. By coordinating data flow and AI-driven processing, the backend plays a central role in enabling reliable, scalable, and data-driven operation of the PredictiX application.

3.3.1 Render

Render is a cloud-based deployment platform that simplifies the hosting and management of backend services. It provides automatic deployment from version control, environment variable management, and built-in support for scalable web services, making it suitable for applications that require continuous availability and reliable performance.

In the PredictiX system, Render is used to deploy and run the FastAPI backend that handles asset

management operations, predictive maintenance inference, maintenance cost estimation, and AI-driven workflows. Hosting the backend on Render ensures that machine learning models, SHAP based explanation processes, and API services remain available without interruption. Render's managed infrastructure reduces operational complexity and allows the development process to focus on system functionality rather than server maintenance, making it a practical choice for deploying a scalable and production-ready backend for PredictiX.

3.3.2 FastAPI

FastAPI is a modern Python-based framework for developing high-performance RESTful APIs, offering built-in asynchronous support, automatic API documentation through Swagger, and a robust dependencyastAPI is a modern Python framework used to build high-performance backend APIs with support for asynchronous processing and automatic API documentation. Its clean structure and scalability make it well suited for applications that need to handle multiple requests efficiently while integrating with data-driven services.

In the PredictiX system, FastAPI acts as the main backend framework, managing communication between the frontend, database, and AI components. It is responsible for handling asset management operations, maintenance ticket workflows, predictive maintenance and cost estimation requests, and report generation. FastAPI's ability to process concurrent requests allows the system to support real-time dashboard updates, chatbot interactions, and prediction queries smoothly.

FastAPI also integrates easily with machine learning pipelines, enabling the deployment of XGBoost models, SHAP based explanations, and NLP services used in the RAG pipeline. Built in support for secure mechanisms such as JWT authentication further ensures secured access to system resources. Overall, FastAPI provides a reliable and scalable backend foundation for the PredictiX application.

3.4 Database Technologies

The system uses a structured database to store core entities such as users, assets, tickets, maintenance history, and system logs. Asset-related data includes asset metadata, maintenance schedules, health scores, and status information. Ticket-related data stores descriptions, priorities, categories, status updates, and comments. Time-series sensor data and asset logs are stored in a scalable manner to support predictive analytics and historical trend analysis.

3.4.1 Supabase

Supabase is an open-source backend platform built on top of PostgreSQL, designed to simplify backend development while maintaining flexibility, reliability, and scalability. It provides essential backend services such as authentication, real-time database updates, file storage, and role-based access control through a unified API. In the PredictiX system, Supabase is used to manage structured warehouse data, including asset records, maintenance histories, user accounts, and maintenance tickets, ensuring secure data storage and controlled access.

In addition to managing relational data, Supabase is also utilized to store vector embeddings generated from maintenance logs, ticket summaries, and asset-related textual information. By

enabling the PostgreSQL pgvector extension, semantic representations of text data can be stored and queried efficiently within the same database environment. This capability supports similarity-based retrieval and retrieval-augmented generation (RAG), allowing the system to retrieve relevant historical records when generating AI-driven reports and chatbot responses.

Supabase's seamless integration with modern frontend frameworks enables real-time synchronization of asset and ticket updates, improving system responsiveness and user experience. By leveraging PostgreSQL as its core database engine, Supabase combines enterprise-grade reliability with developer-friendly tooling, making it a suitable choice for implementing a scalable, cost-effective warehouse management solution that supports both structured data management and advanced AI-driven retrieval.

3.5 AI Technologies

The PredictiX application uses multiple AI techniques to support predictive maintenance and intelligent decision-making. Machine learning models analyze historical maintenance records, asset metadata, and available sensor data to predict equipment failures, estimate maintenance costs, and generate asset health scores with associated confidence levels. Explainable AI (XAI) methods such as SHAP are applied to interpret these predictions by identifying key factors influencing model outputs.

Natural Language Processing techniques are used to automatically categorize maintenance tickets, predict ticket priority, and generate concise ticket summaries from free text descriptions. Retrieval augmented generation (RAG) is used to produce data driven reports and chatbot responses by combining predictive outputs with relevant historical records. An AI-powered chatbot enables users to access asset status, maintenance schedules, predictions, cost estimates, and ticket information through natural language interaction.

3.5.1 Predictive Maintenance and Failure Prediction (XGBoost)

PredictiX utilizes gradient boosted decision tree models based on XGBoost to perform predictive maintenance for warehouse assets. The model analyzes structured historical data, including asset metadata, maintenance logs, operational indicators, and available sensor-derived features, to estimate asset health scores and predict the probability of equipment failure within a given time period. By learning non linear relationships and feature interactions, the model enables early identification of high risk assets, supporting proactive maintenance planning and reducing unexpected downtime.

3.5.2 Maintenance Cost Estimation (XGBoost)

In addition to failure prediction, PredictiX uses a separate XGBoost based model to estimate potential maintenance costs associated with predicted failures. This model considers factors such as asset type, historical repair patterns, failure indications, downtime sensitivity, and maintenance history. The resulting cost estimates allow maintenance teams to prioritize actions based not only on failure risk but also on expected financial impact, enabling more cost aware and efficient resource allocation.

3.5.3 Prediction Explainability (SHAP)

To ensure transparency and trust in AI-driven predictions, PredictiX integrates SHAP (SHapley Additive exPlanations) as its explainable AI framework. SHAP is used to interpret both failure prediction and maintenance cost estimation models by quantifying the contribution of individual input features to each prediction. This enables users to understand why an asset is classified as high risk or why a particular cost estimate is generated, supporting informed decision making and increasing confidence in model outputs.

3.5.4 Maintenance Ticket Categorization and Priority Prediction (DistilBERT)

PredictiX applies natural language processing techniques using the DistilBERT transformer model to analyze free text maintenance tickets. Ticket titles and descriptions are processed to automatically categorize tickets and predict their priority levels (low, medium, or high). This automation reduces manual effort, improves consistency in ticket handling, and ensures that critical issues are identified and addressed in a timely manner.

3.5.5 Maintenance Ticket Summarization (DistilBART)

To improve clarity and traceability in ticket management, PredictiX uses an abstractive summarization model based on DistilBART. The model generates concise summaries from ticket descriptions, updates, and resolution notes, capturing key issues and actions taken. These summaries help maintenance teams quickly understand ticket context and are also reused as structured inputs for reporting and retrieval based analytics.

3.5.6 Semantic Retrieval and Knowledge Grounding (Sentence Transformers and RAG)

PredictiX uses sentence transformer based embedding models to convert maintenance logs, ticket summaries, and asset descriptions into vector representations. These embeddings are stored using a vector database and enable semantic similarity search. Retrieval Augmented Generation (RAG) is then used to retrieve relevant historical records when generating reports or responding to user queries. This approach ensures that AI generated insights and explanations are grounded in actual system data rather than generic or unverified information.

3.5.7 Report Generation and Validation (RAG and LLM with Multi Step Agents)

PredictiX supports automated generation of asset-level and warehouse-level maintenance reports using a multi-step, agent-based AI pipeline. The report generation process begins by retrieving relevant historical maintenance records, ticket summaries, and asset information through retrieval augmented generation (RAG). Predictive maintenance and cost estimation outputs are then incorporated, along with SHAP based explanations that highlight key factors influencing model predictions.

To ensure reliability, retrieved information and generated insights are validated against stored system data before report finalization. The use of a multi step agent workflow enables structured data retrieval, reasoning, and content generation, ensuring that reports are accurate, explainable, and grounded in verified evidence. This approach supports strategic maintenance planning, improves

transparency, and provides actionable insights for warehouse level operational analysis.

3.5.8 AI Powered Chatbot (RAG with LLM)

The chatbot component is designed as an intelligent interface that allows users to interact with the system using natural language. Its main purpose is to explain asset risk predictions, answer operational questions, retrieve relevant knowledge, and suggest appropriate actions. To support these functions, the chatbot integrates a Large Language Model , Retrieval-Augmented Generation (RAG), and Shapley Additive explanations(SHAP).

Instead of operating as a single process, the chatbot follows an agent based design with three logical roles. planning, tool execution, and response generation. These roles are implemented within a single FastAPI service using one LLM, allowing the system to remain lightweight while supporting agent-like behavior.

The first role acts as a planning agent. It analyzes user queries, identifies user intent, and determines which system components need to be accessed to fulfill the request.

The second role operates as a tool execution agent. This component retrieves factual data from Supabase, including asset information and prediction results. It also uses RAG to obtain relevant documents such as standard operating procedures or previously resolved cases. This role focuses only on collecting structured information from the system.

The final role is the Response Generator Agent. It receives the collected results and retrieved documents, then produces clear and user-friendly explanations. Asset risk levels are explained, and recommended actions are supported with related documentation. The LLM is restricted to using only retrieved information to ensure accurate and reliable responses.

Conceptually, these roles function as separate agents; however, they are implemented as role-based logic within a single chatbot endpoint. This approach simplifies deployment while enabling dynamic planning, tool usage, and grounded response generation.

This architecture allows the chatbot to combine predictive analytics with contextual knowledge retrieval, providing transparent explanations and practical recommendations. As a result, the chatbot serves as an intelligent assistant rather than a basic question-answer system.

3.5.9 AI-Powered Chatbot and Multi-Agent Interaction (RAG with LLM)

Each agent plays a specific role, allowing the system to process questions in a coordinated and organized way. When a user asks a question, the system distributes tasks among the agents and gathers the required information from internal data sources and related documents. Retrieval-Augmented Generation (RAG) helps bring in relevant knowledge, while the Large Language Model (LLM) combines everything into a meaningful response. This collaborative approach allows the chatbot to manage multi-step tasks smoothly while keeping responses accurate and consistent. By using agent cooperation with RAG and LLM, the chatbot provides helpful insights

and practical recommendations, supporting users in making better operational decisions rather than simply answering questions.

3.6 Software Development Process Model

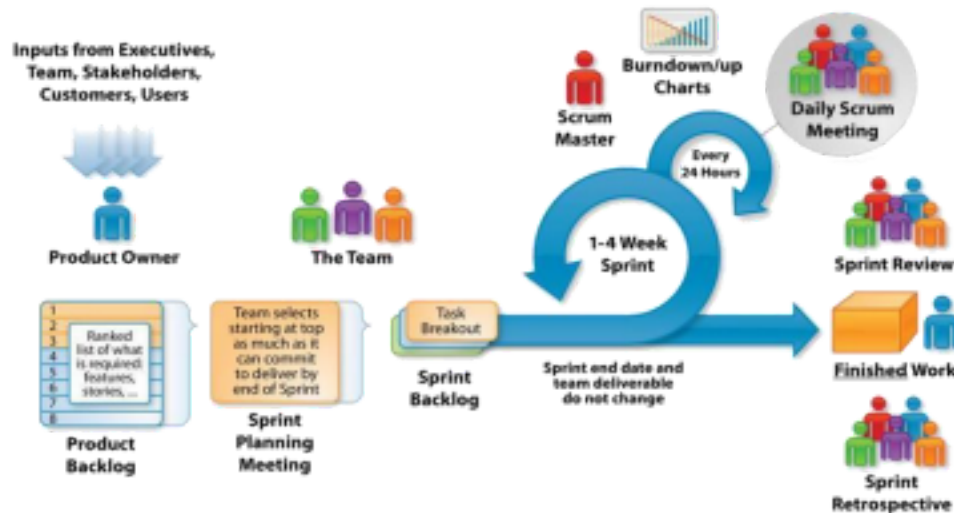


Figure 3.6 1: Software Development Process Model- Agile Methodology

The project began with a meeting involving both industry representative and academic supervisor to clearly identify the core requirements of the warehouse asset management and predictive maintenance system. During this phase, the main objectives like predictive maintenance, ticket management, and role-based access were finalized. To ensure transparency and alignment, cloud based documentation including user stories and the Software Requirements Specification were discussed among all stakeholders. This early alignment helped define clear deliverables and provided a strong foundation for structured system development.

After detailed discussions and requirement analysis, the team selected the Agile Scrum Model as the software development process model. This choice was driven by the project's need to handle evolving requirements, particularly in areas such as AI-driven predictions, dashboard insights and user experience improvements. Agile Scrum supports iterative development, frequent reviews, and continuous feedback, allowing the system to grow incrementally while maintaining quality .

To maintain consistent progress and ensure the system meets expectations, weekly or bi-weekly review meetings were conducted with the academic supervisor and, where applicable, industry mentors. These sessions were used to present completed features, review outputs, discuss challenges, and validate design decisions. Regular feedback helped identify improvements early, reduced rework,

and ensured the project remained aligned with real-world warehouse operational needs.

The development work was divided into sprints, each focusing on high-priority functionalities such as asset management, predictive maintenance visualization, ticket lifecycle handling, and warehouse-level analytics. By delivering functional components at the end of each sprint, the team was able to gather feedback continuously and adapt the system accordingly. This iterative refinement supported improvements in UI/UX design, optimization of workflows, and enhancement of dashboard features.

User stories were used extensively to manage the sprint backlog and clearly define system behavior from an end-user perspective. Core functionalities such as viewing asset health, creating maintenance tickets, monitoring predictions, and generating reports were broken down into manageable development tasks. This approach ensured clarity in sprint goals, improved team coordination, and enabled efficient tracking of progress throughout the development lifecycle.

3.7 User Role, Input- Process - Output of the System

3.7.1 User Role

Role	Activities
Admin	● Log in to the system with administrative privileges ● Manage user accounts and role based access permissions ● Create, update, and manage warehouse asset records ● View predictive maintenance insights and SHAP based explanations ● Generate asset level and warehouse level maintenance reports ● Monitor warehouse level analytics and dashboards
Standard User	● Log in to the system with user level access ● Create, update, and track maintenance tickets ● View asset details and maintenance history ● Monitor the condition and health status of assigned assets ● Review performance indicators and ticket-related updates

3.7.2 Inputs - Process - Output

Inputs

- Asset metadata, operational status, and maintenance schedules
- Historical maintenance records and available sensor readings
- Free text maintenance ticket titles and descriptions
- User profiles and role-based access information

Processes

- Data validation, preprocessing, and secure storage
- Predictive maintenance and maintenance cost estimation analysis
- Automated ticket categorization, prioritization, and summarization
- Retrieval-augmented report generation and dashboard updates
- Notification handling and AI chatbot query processing

Outputs

- Asset health scores, failure probability predictions, and cost estimates
- Categorized and prioritized maintenance tickets with summaries
- Interactive dashboards and data-driven analytical reports
- Alerts and notifications for critical assets and maintenance events

3.8 Processes

The system follows a structured set of processes that transform raw operational data into meaningful insights and actionable decisions. The process begins with asset registration, where administrators create and maintain asset records, including descriptions, maintenance schedules, and historical logs. Asset logs and readings play a crucial role, as predictive maintenance functionality depends on the availability of accurate historical data.

Once asset data is available, the predictive analytics process evaluates asset health scores, failure probabilities, cost estimations and predicted maintenance dates. These predictions are visualized through charts and trends, allowing users to identify assets that require immediate attention. Based on predicted or confirmed issues, maintenance tickets are generated, categorized, prioritized, and tracked

throughout their lifecycle until resolution.

At a higher level, the system aggregates asset and ticket data to generate warehouse insights, such as overall health percentages, asset status distributions, and ticket trends. Reports are generated at both asset and warehouse levels to support management decisions. Throughout all processes, notifications and summaries ensure transparency, accountability, and efficient collaboration between users and administrators.

3.9 Our Approach

We present PredictiX: an AI-driven Warehouse Asset Management and Predictive Maintenance Platform, designed to support proactive maintenance operations and informed decision making in warehouse environments. The platform combines intelligent analytics, role-based system management, and intuitive visualizations to improve asset reliability and maintenance efficiency. The proposed approach emphasizes practical usability, data accuracy, and scalability, ensuring that the system can be effectively deployed and adapted in real operational settings.

Artificial intelligence plays a central role in enhancing the capabilities of PredictiX. Predictive maintenance and maintenance cost estimation are performed using tabular machine learning models that analyze historical maintenance records, asset metadata, and available sensor data. These models generate asset health scores, failure probability forecasts, estimated maintenance costs, and prediction confidence levels. Natural language processing techniques are used to automatically categorize, prioritize, and summarize maintenance tickets from free text descriptions. Retrieval augmented generation (RAG) is used to produce data grounded reports and chatbot responses by combining predictive outputs with relevant historical records, while explainable AI techniques provide transparency by highlighting the key factors influencing each prediction.

The system is developed using an iterative and modular development approach, allowing individual components such as asset management, ticket handling, predictive analytics, dashboards, and chatbot functionality to evolve independently. Incremental development and continuous refinement ensure that system features remain aligned with operational requirements and user expectations. This modular design also supports easier maintenance, testing, and future system extensions.

Our solution, PredictiX, clearly defines user responsibilities through role-based access control. Administrators are responsible for managing users, assets, predictive insights, and warehouse level reports, while standard users can submit and track maintenance tickets, monitor assigned assets, and review system generated insights. This separation of roles ensures operational clarity, secure access, and efficient system usage across different user groups.

To ensure security and scalability, PredictiX incorporates secure authentication mechanisms, role-based authorization, and encrypted communication protocols. The cloud-based architecture supports concurrent users and real-time data access, while the backend efficiently handles AI workloads, predictive computations, and report generation. By combining predictive analytics, explainable AI, intelligent automation, and scalable system design, PredictiX delivers a reliable and intelligent solution that enhances modern warehouse asset management and maintenance planning.

Chapter 4: Analysis and Design

4.1 Introduction

This chapter focuses on the analysis and design aspects of the AI-powered Warehouse Asset Management and Predictive Maintenance System. Before implementation, it is essential to clearly understand system requirements, user expectations, and operational constraints. The analysis phase ensures that the proposed solution addresses real warehouse challenges such as asset failures, maintenance delays, inefficient tracking of maintenance activities, and limited visibility into asset health and maintenance costs.

Based on the gathered requirements, the design phase translates both functional and non-functional needs into a structured system architecture. This includes user role definitions, data flow between system components, database design, interface layouts, and the integration of predictive analytics and AI-driven services. Special attention is given to explainability, data validation, and secure access control to ensure that AI-generated insights are transparent and trustworthy.

A well planned design helps ensure system scalability, usability, and reliability while minimizing implementation risks. By clearly defining system interactions and responsibilities at this stage, the design provides a strong foundation for integrating machine learning models, intelligent ticket management, reporting mechanisms, and chatbot-based user interactions in later phases of development.

4.2 Analysis

4.2.1 Functional Requirements

4.2.1.1 User Registration and Authentication

- **REQ-1:** The system must allow users to authenticate using a registered email address and password.
- **REQ-2:** The system must support secure user authentication using JSON Web Tokens (JWT).
- **REQ-3:** The system must prevent duplicate user accounts by ensuring that a single email address cannot be registered more than once.
- **REQ-4:** The system must support role-based access control for administrators and standard users.
- **REQ-5:** The system must allow only administrators to create new user and administrator accounts through the user management section.
- **REQ-6:** The system must allow only administrators to change details of user accounts.

- **REQ-7:** Standard users must not be permitted to create, modify, or delete user accounts.

4.2.1.2 Asset Creation and Management

- **REQ-6:** The system must allow administrators to create, update, view, and delete warehouse asset records.
- **REQ-7:** The system must store and display asset details including asset type, operational status, maintenance schedules, and historical maintenance records.
- **REQ-8:** The system must allow standard users to view assets assigned to them along with relevant maintenance and condition information.

4.2.1.3 Predictive Maintenance and Cost Estimation

- **REQ-9:** The system must analyze historical maintenance data and available sensor readings to calculate asset health scores.
- **REQ-10:** The system must predict asset failure probabilities over a defined future time window.
- **REQ-11:** The system must estimate potential maintenance costs based on asset characteristics and predicted failure risks.
- **REQ-12:** The system must provide confidence indicators or supporting metrics for predictive maintenance and cost estimation outputs.

4.2.1.4 Maintenance Ticket Management

- **REQ-13:** The system must allow users to create and update maintenance tickets using free-text descriptions.
- **REQ-14:** The system must automatically categorize maintenance tickets using natural language processing techniques.
- **REQ-15:** The system must predict maintenance ticket priority levels (low, medium, high) based on ticket content.
- **REQ-16:** The system must generate concise summaries of maintenance tickets, along with summarized related asset logs, to improve readability and tracking.

4.2.1.5 AI-Powered Chatbot Interaction

- **REQ-17:** The system must support chatbot-based querying using natural language input.
- **REQ-18:** The chatbot must allow users to retrieve asset information, maintenance schedules,

predictive insights, cost estimates, tickets, and reports.

- **REQ-19:** The chatbot must provide responses grounded in verified system data, historical records, and predictive model outputs.
- **REQ-20:** The chatbot must enforce role-based access control, ensuring that administrators can access warehouse-level insights and report generation features, while standard users are restricted to asset- and ticket-level information only.

4.2.1.6 Reporting and Visualization

- **REQ-21:** The system must generate asset-level and warehouse-level maintenance reports.
- **REQ-22:** The system must display real-time visualizations of asset health scores, failure probability trends, estimated maintenance costs, and ticket statuses.
- **REQ-23:** The system must provide dashboard views for monitoring warehouse performance, critical assets, and maintenance workload.

4.2.1.7 Explainability and Transparency

- **REQ-24:** The system must provide explainable AI outputs that highlight key factors influencing failure predictions and maintenance cost estimates.
- **REQ-25:** The system must present prediction explanations in a form understandable to non-technical users, using clear descriptions and visual aids where appropriate.

4.2.2 Non-Functional Requirements

- **Performance:** The system should respond to user requests and prediction queries within acceptable time limits, even under concurrent usage.
- **Security:** The system must ensure secure data handling through authentication, authorization, and encrypted communication.
- **Safety:** The system should prevent unauthorized access and ensure data integrity across all operations.
- **Usability:** The system should provide an intuitive and user-friendly interface suitable for non-technical warehouse users.
- **Scalability:** The system should support growth in the number of users, assets, and data volume without significant performance degradation.
- **Maintainability:** The system should be modular and easy to update, test, and extend.

- **Flexibility:** The system should allow future integration of additional AI models, sensors, or reporting features.

4.3 Design

The system design structure improves maintainability by allowing each module to be developed and updated independently without impacting the entire system. Role-based access control ensures that administrators and standard users can access only the features related to their responsibilities, which helps maintain security and avoid unnecessary system complexity.

The user interface is designed with simplicity and ease of use in mind. Dashboards, charts, and filtering options are provided to help users quickly understand asset conditions and maintenance priorities. Visual elements such as health score trends, failure probability forecasts, estimated maintenance costs, and asset status distributions help users make decisions faster. Navigation between assets, tickets, dashboards, and reports is kept straightforward so that users can move through the system without confusion or extra effort.

From a data perspective, the system design supports structured storage of assets, maintenance logs, tickets, and user information, while also allowing smooth integration with AI and NLP models. Predictive maintenance and cost estimation outputs are directly linked to dashboards, reports, and automated ticket prioritization. Explanation features are included to show the main factors influencing predictions, and report and chatbot responses are based on stored system data to avoid inconsistencies. Overall, this design supports a clear flow of data from collection to analysis and insight generation, while remaining flexible enough to support future system expansion.

4.3.1 Use case Diagram

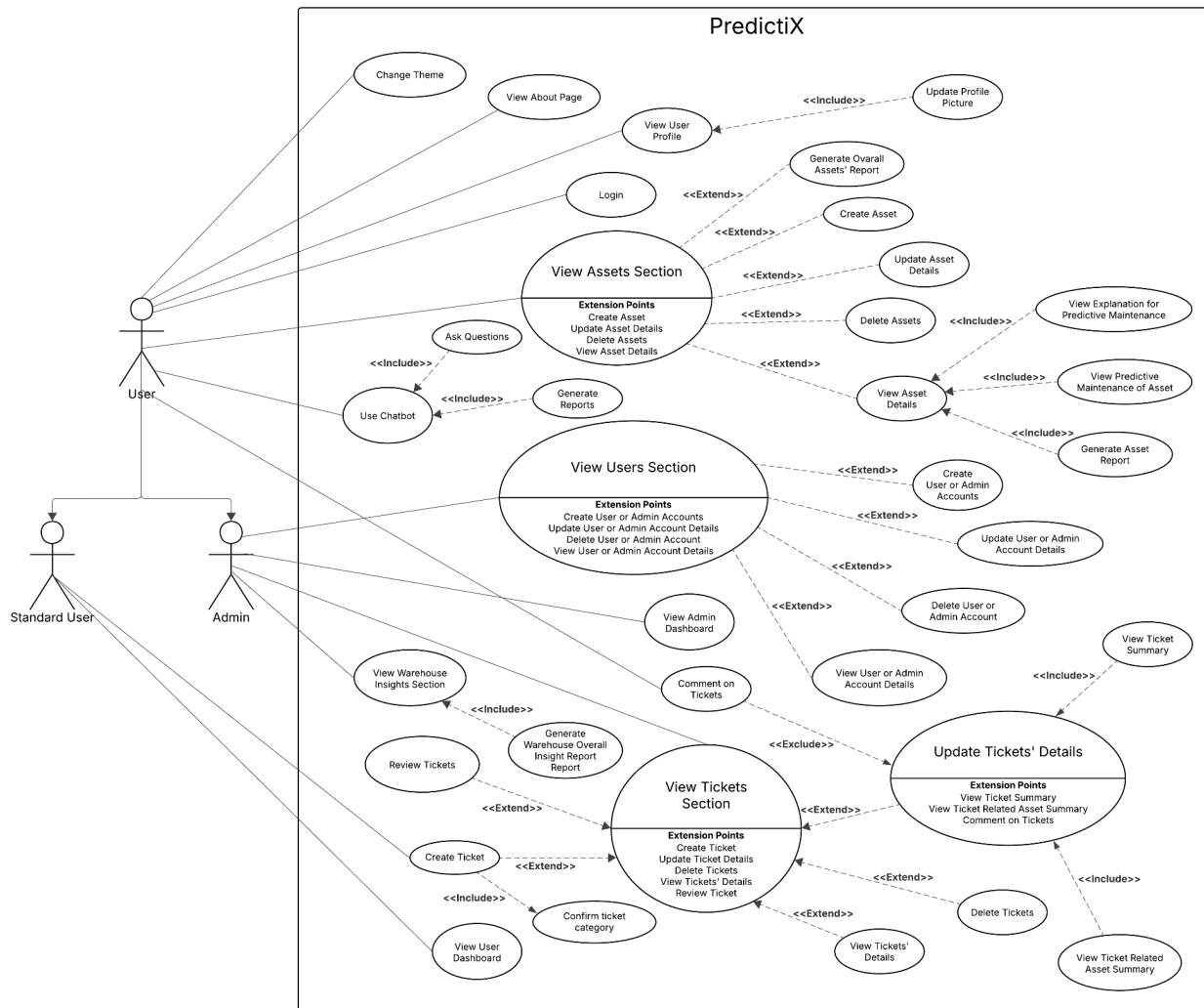


Figure 4.3.1: Use Case Diagram

4.3.2 Activity Diagrams

4.3.2.1 Login

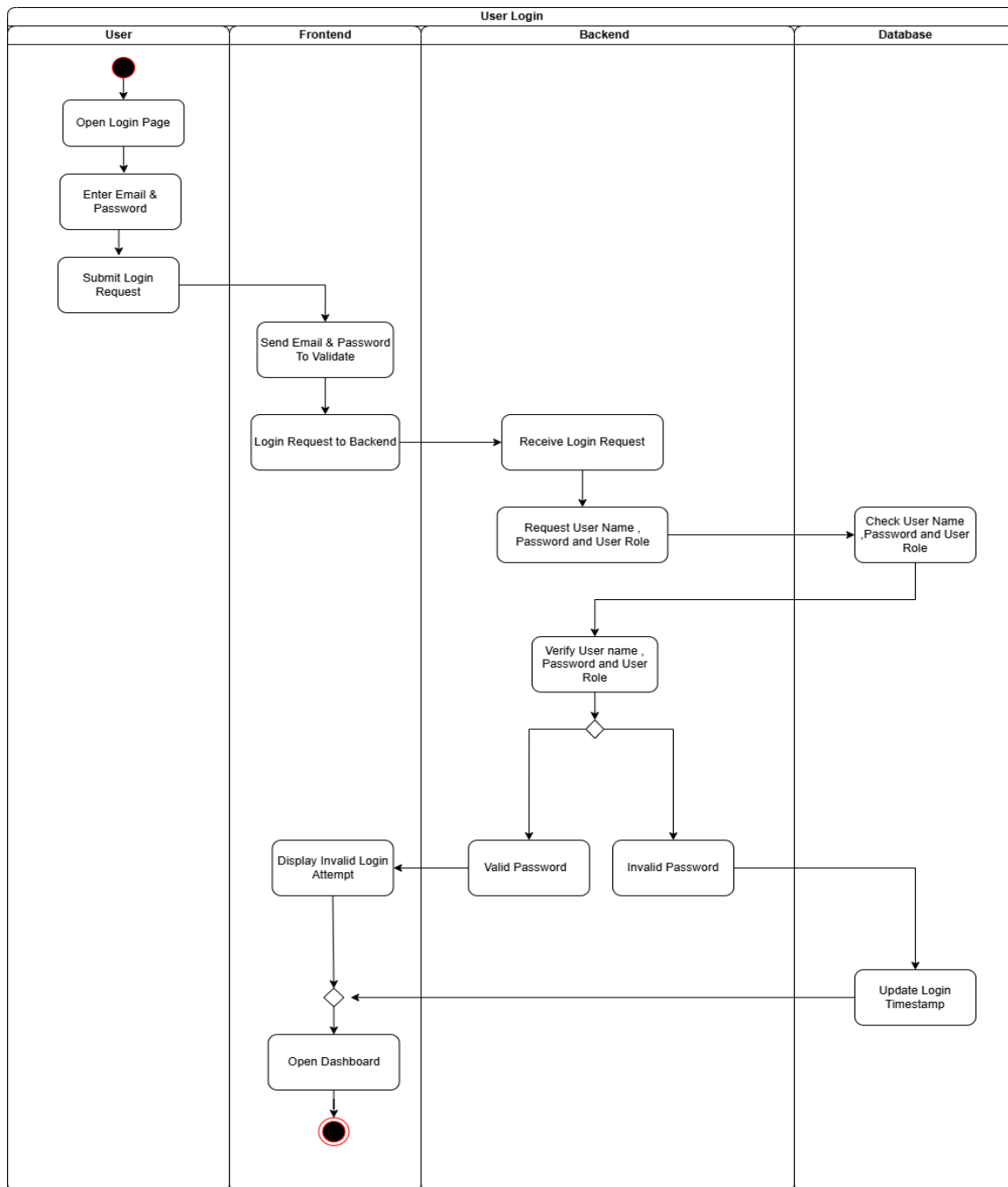


Figure 4.3.2.1: Login Activity Diagram

4.3.2.2 Asset Management (Create, View, Update, Delete Assets)

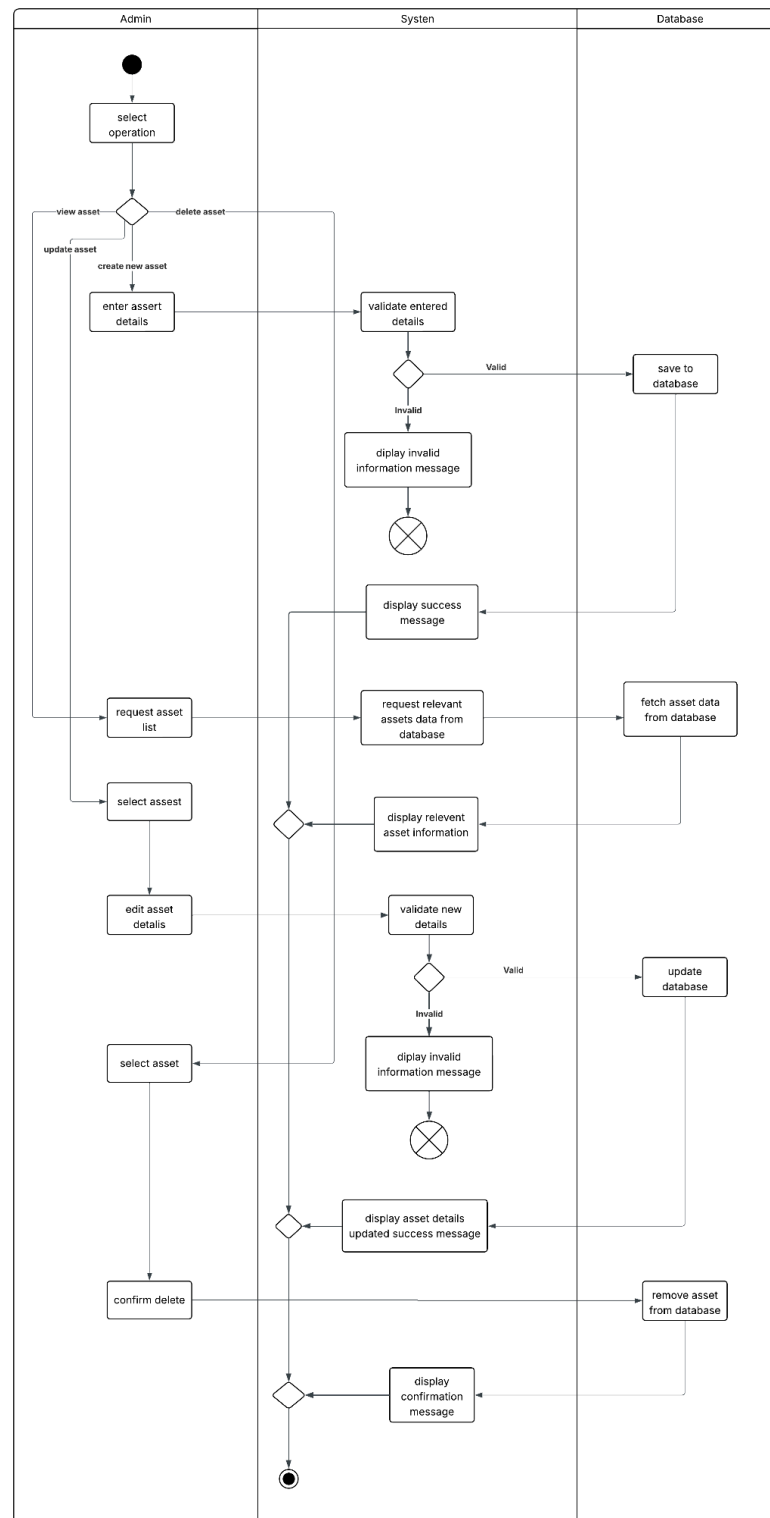


Figure 4.3.2.2: Asset Management Activity Diagram

4.3.2.3 Search Assets

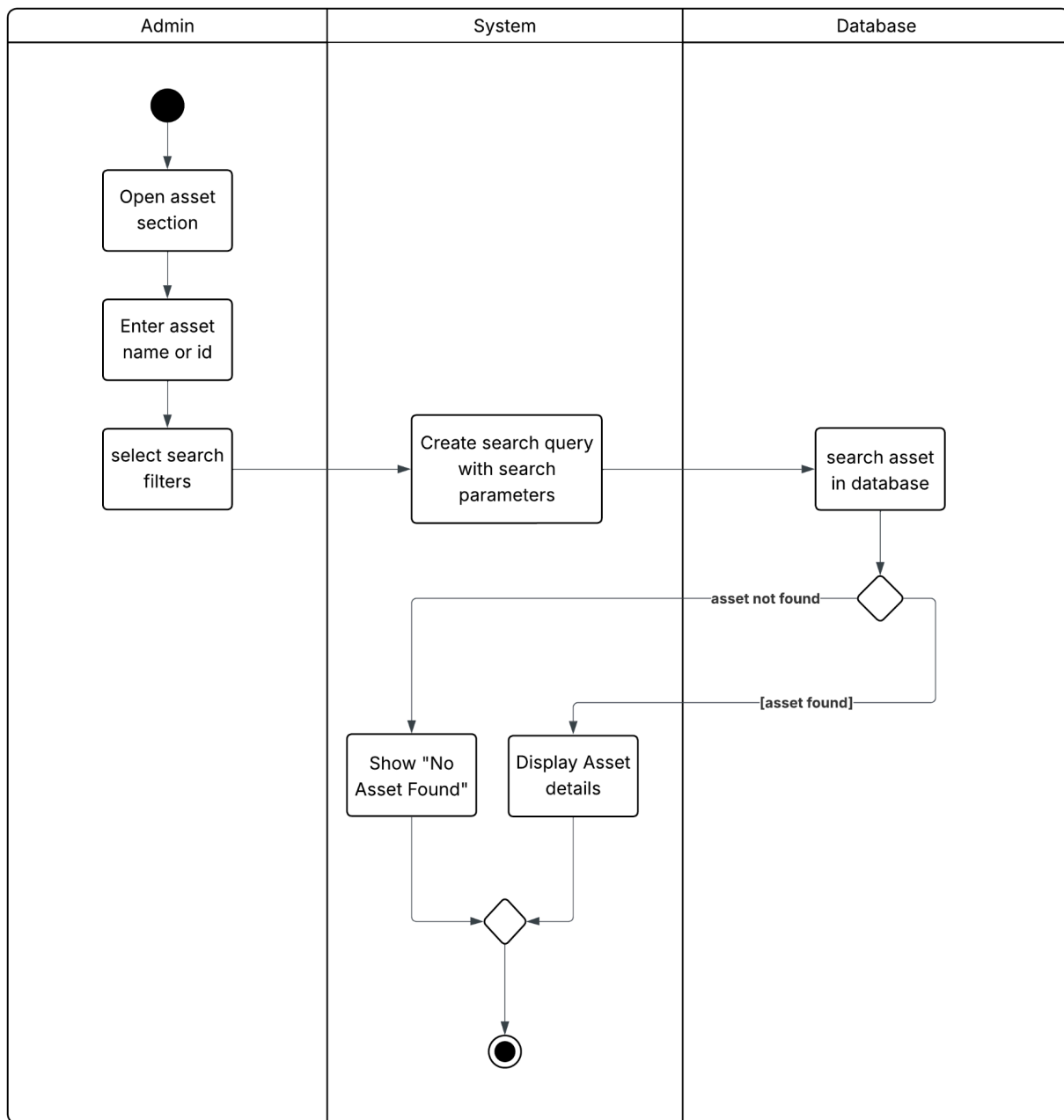


Figure 4.3.2.3: Search Assets Activity Diagram

4.3.2.4 Asset Report Generation

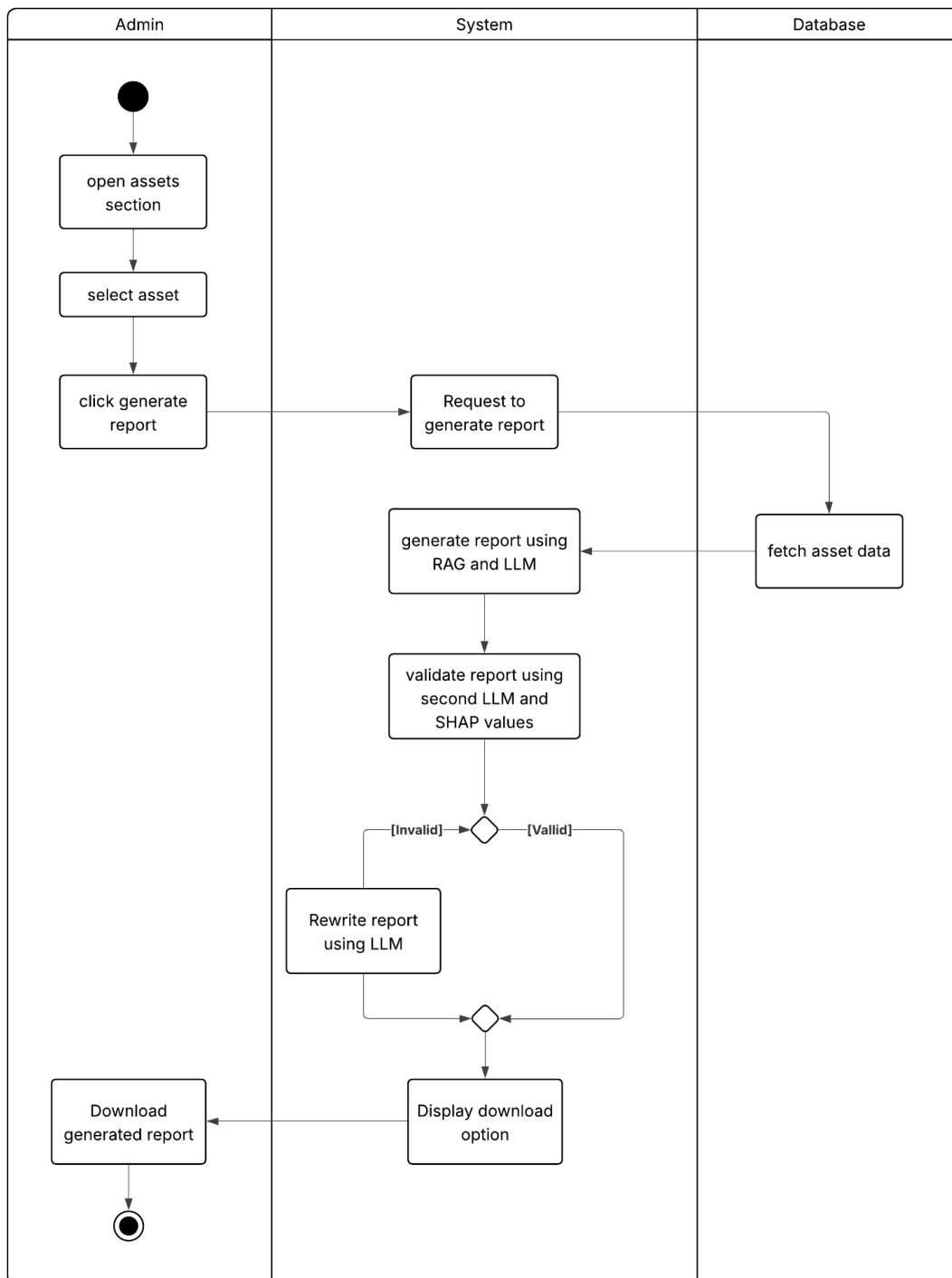


Figure 4.3.2.4: Assets Report Generation

4.3.2.5 Create Ticket

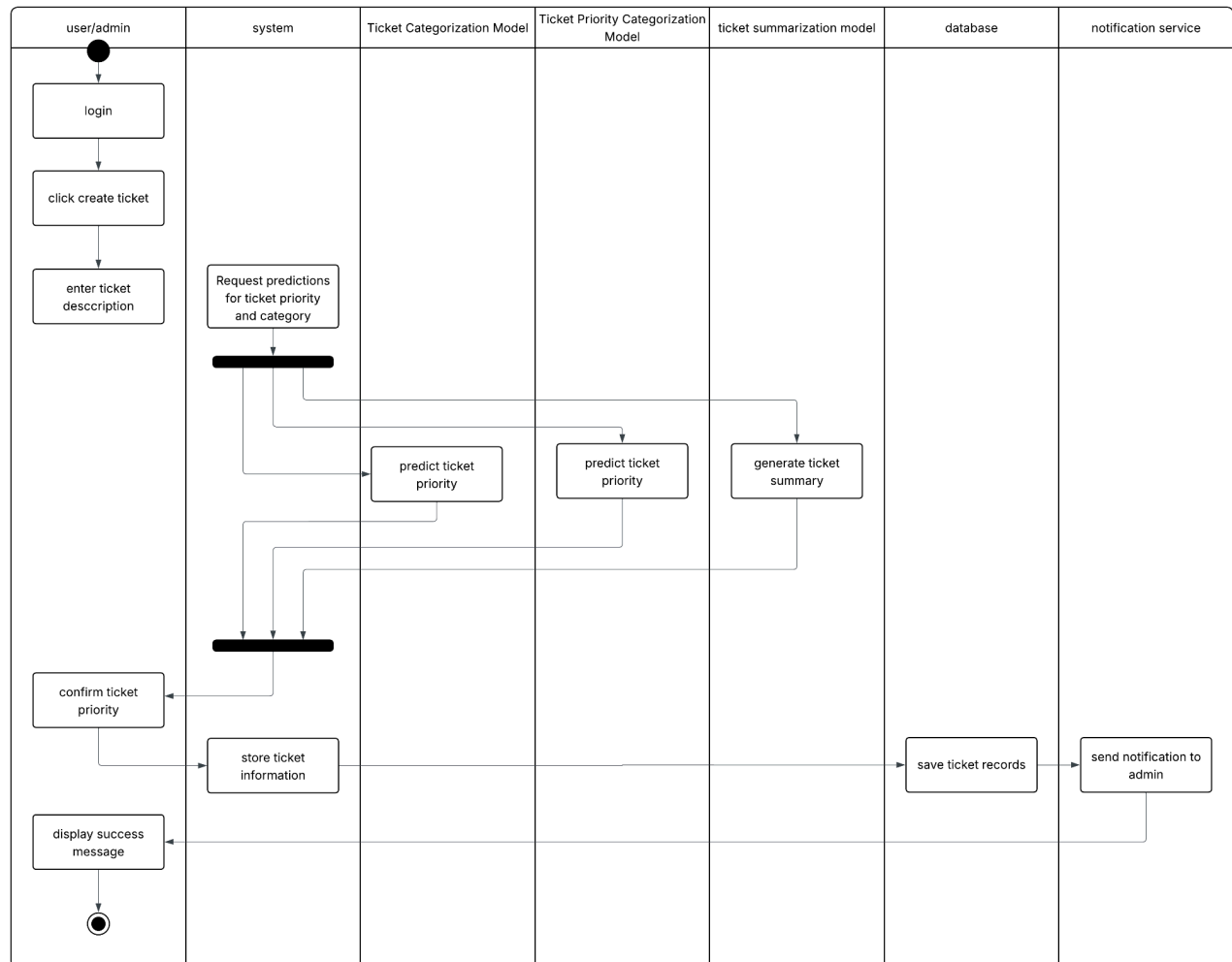


Figure 4.3.2.5 Create Ticket Activity Diagram

4.3.2.6 View, Update, Delete Tickets

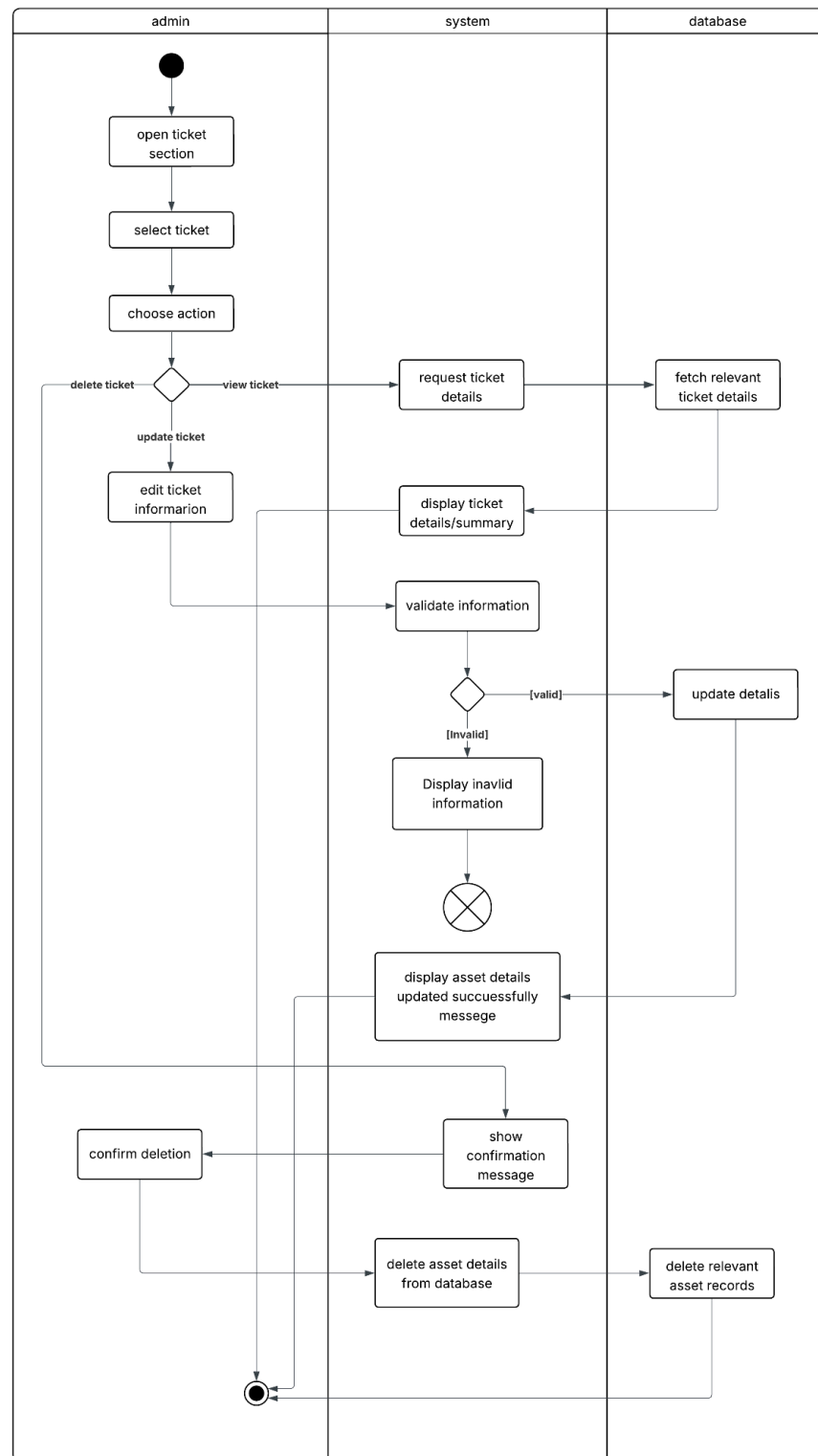


Figure 4.3.2.6 View, Update, Delete Tickets

4.3.2.7 Review Tickets

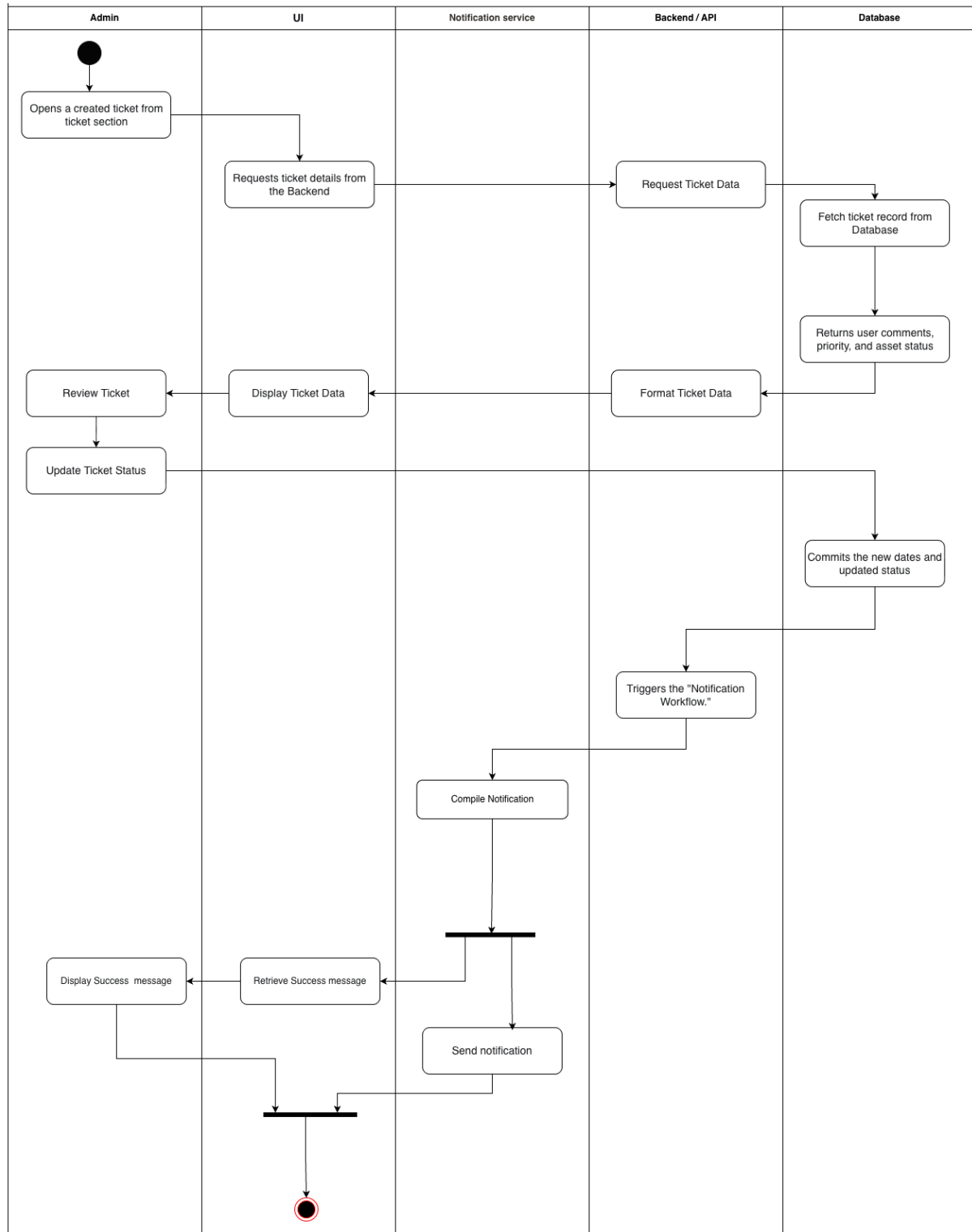


Figure 4.3.2.7 Review Tickets Activity Diagram

4.3.2.8 Search Tickets

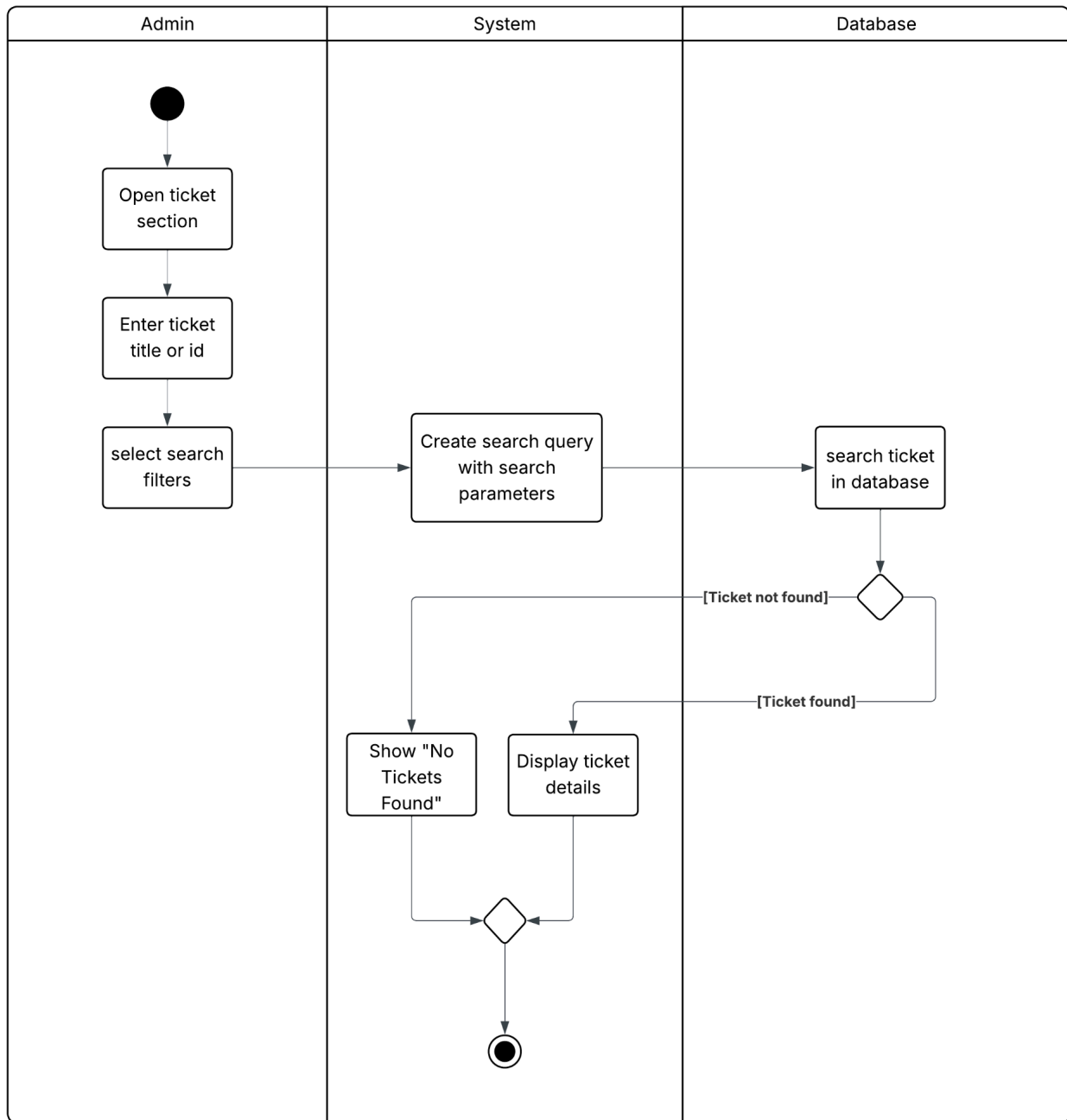


Figure 4.3.2.8 Search Tickets Activity Diagram

4.3.2.9 View Profile

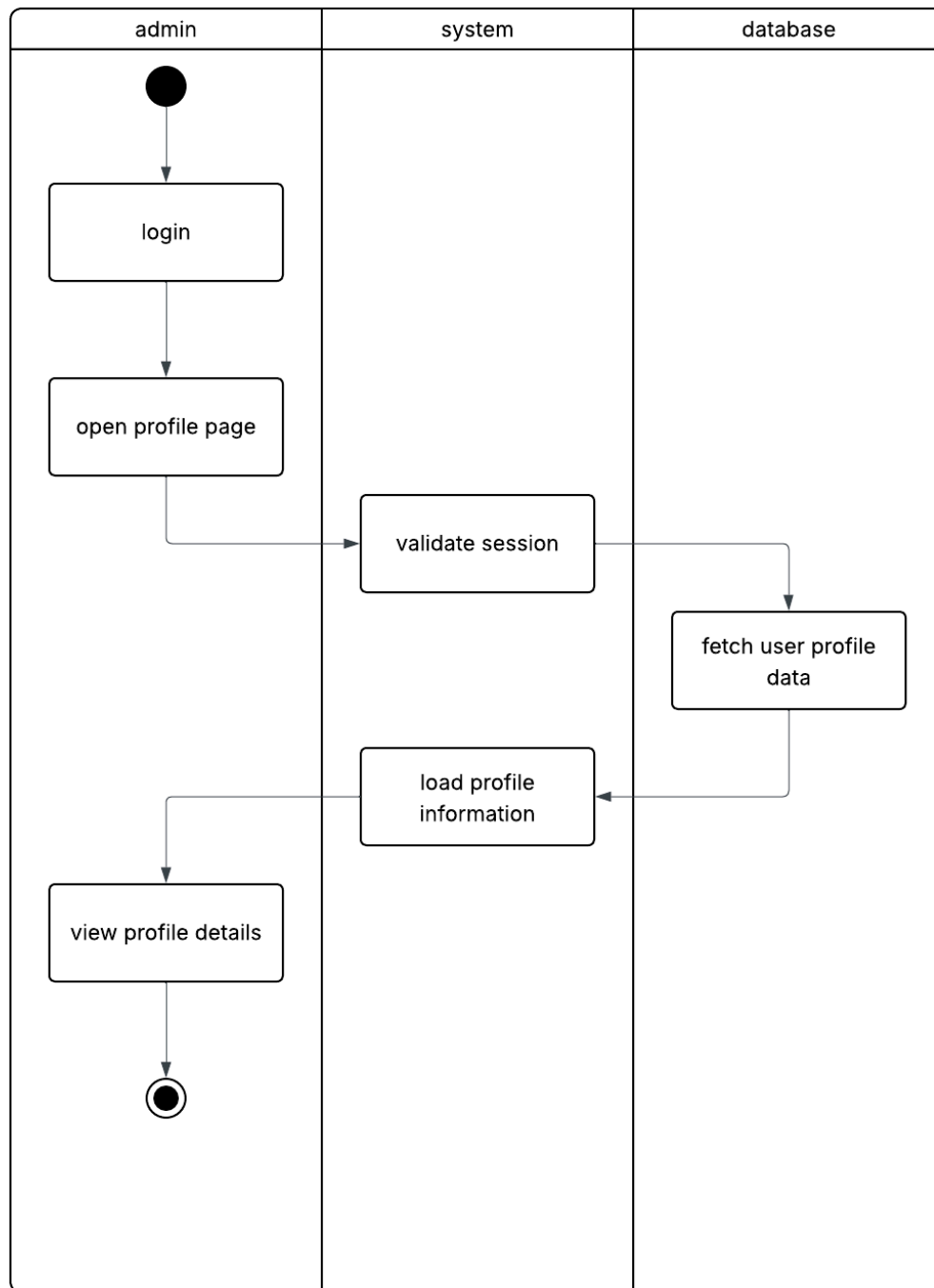


Figure 4.3.2.9 View Profile Activity Diagram

4.3.2.10 Manage Users (Create, View, Update, Delete Users)

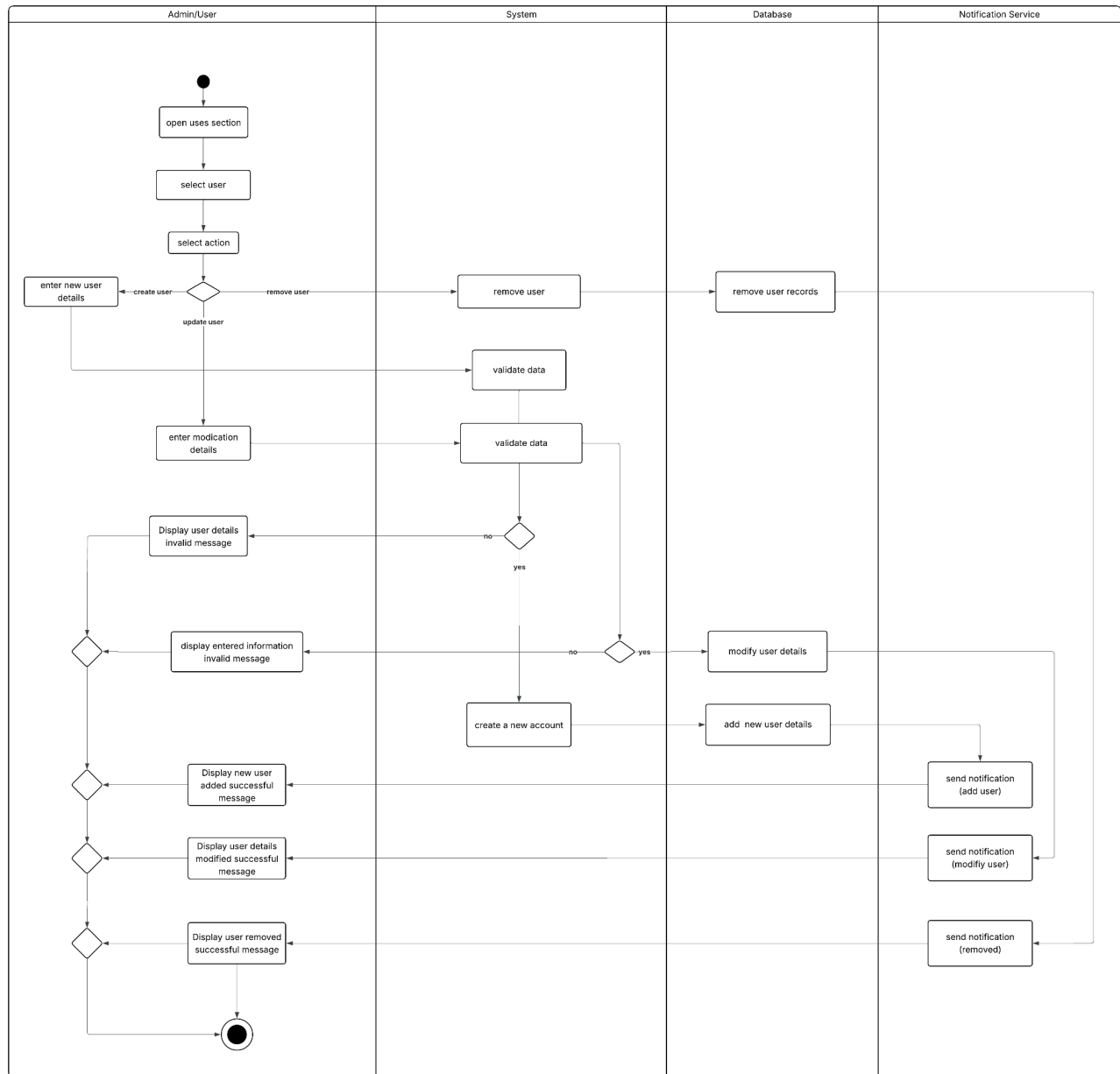


Figure 4.3.2.10 Manage Users (Create, View, Update, Delete Users) Activity Diagram

4.3.2.11 Search Users

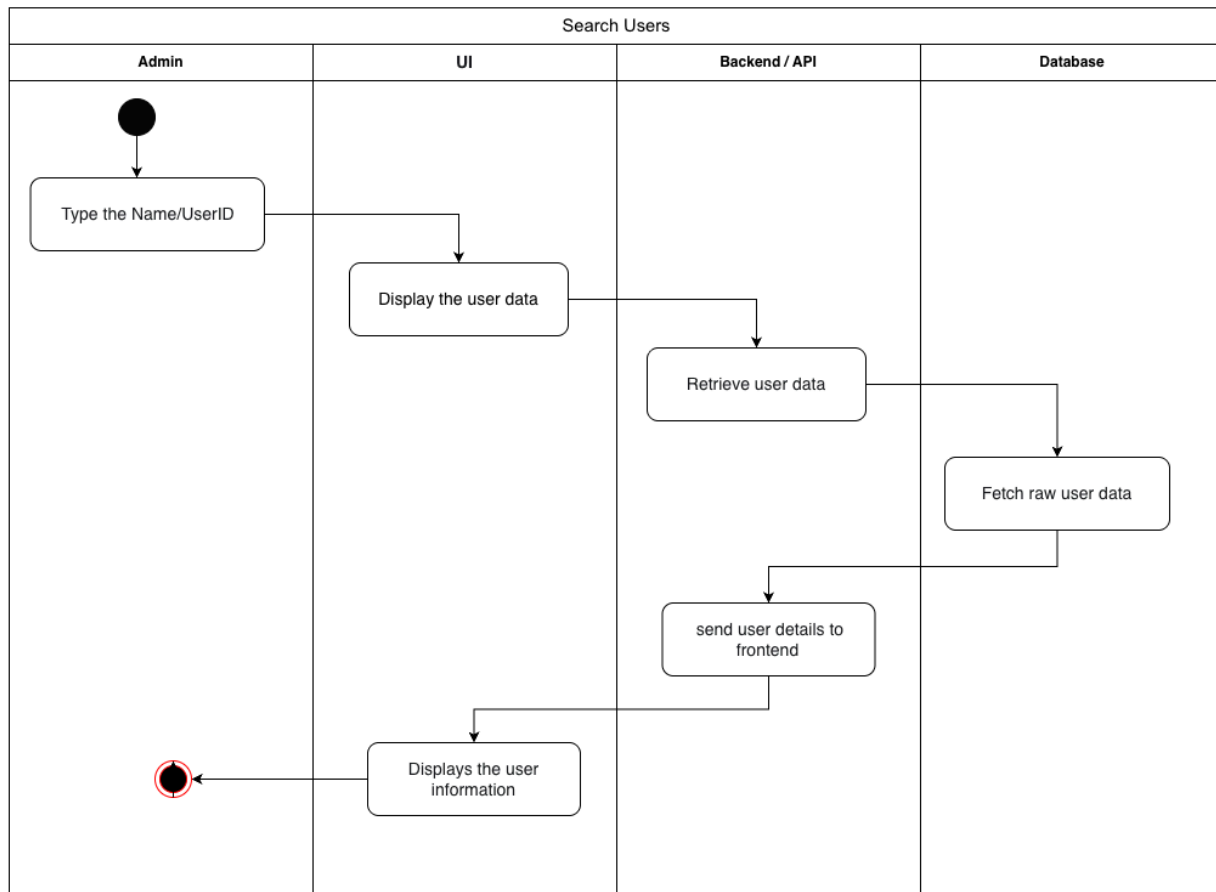


Figure 4.3.2.11 Search Users Activity Diagram

4.3.2.12 View warehouse insights

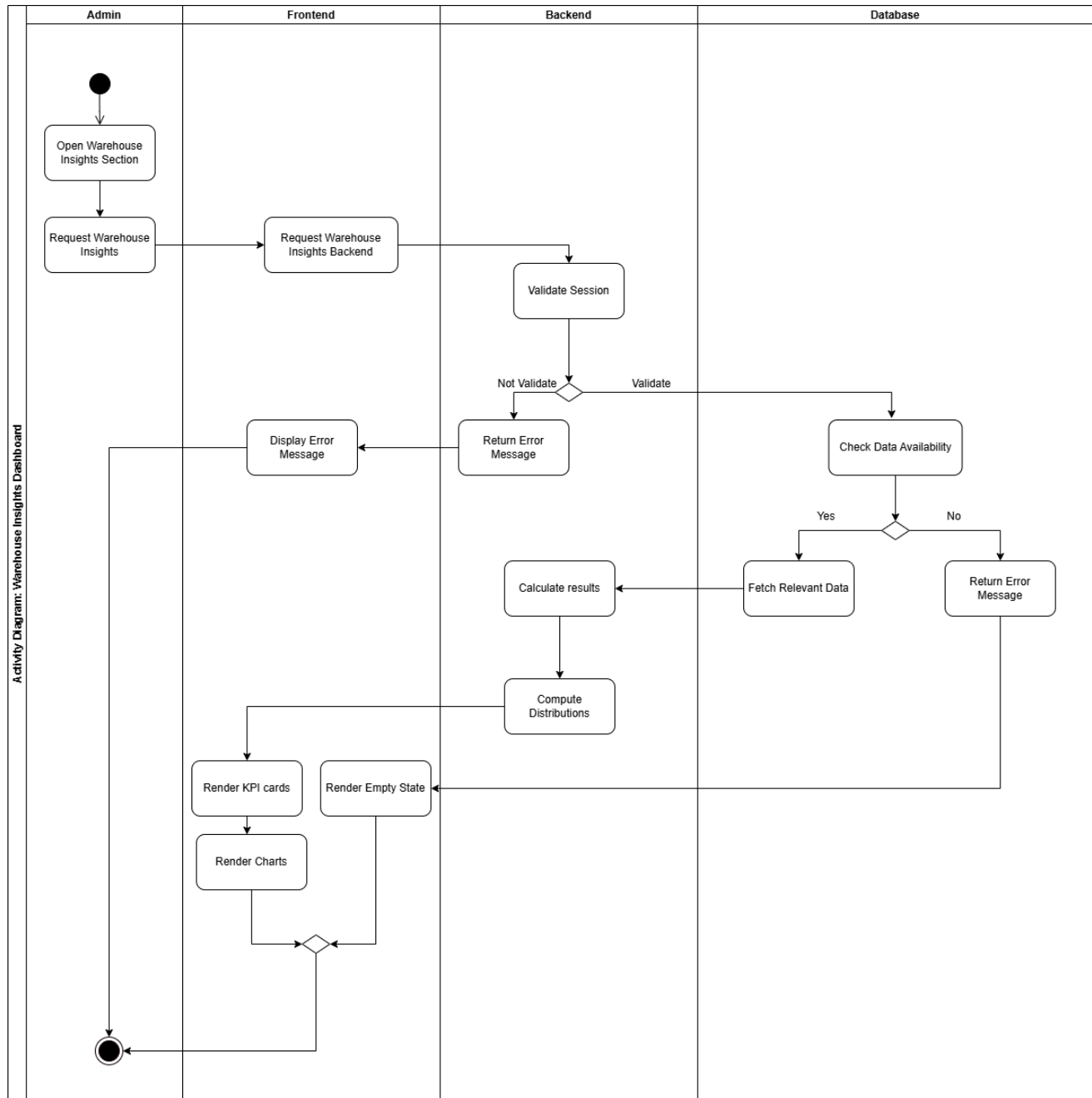


Figure 4.3.2.12 View warehouse insights Activity Diagram

4.3.2.13 Warehouse Report Generation

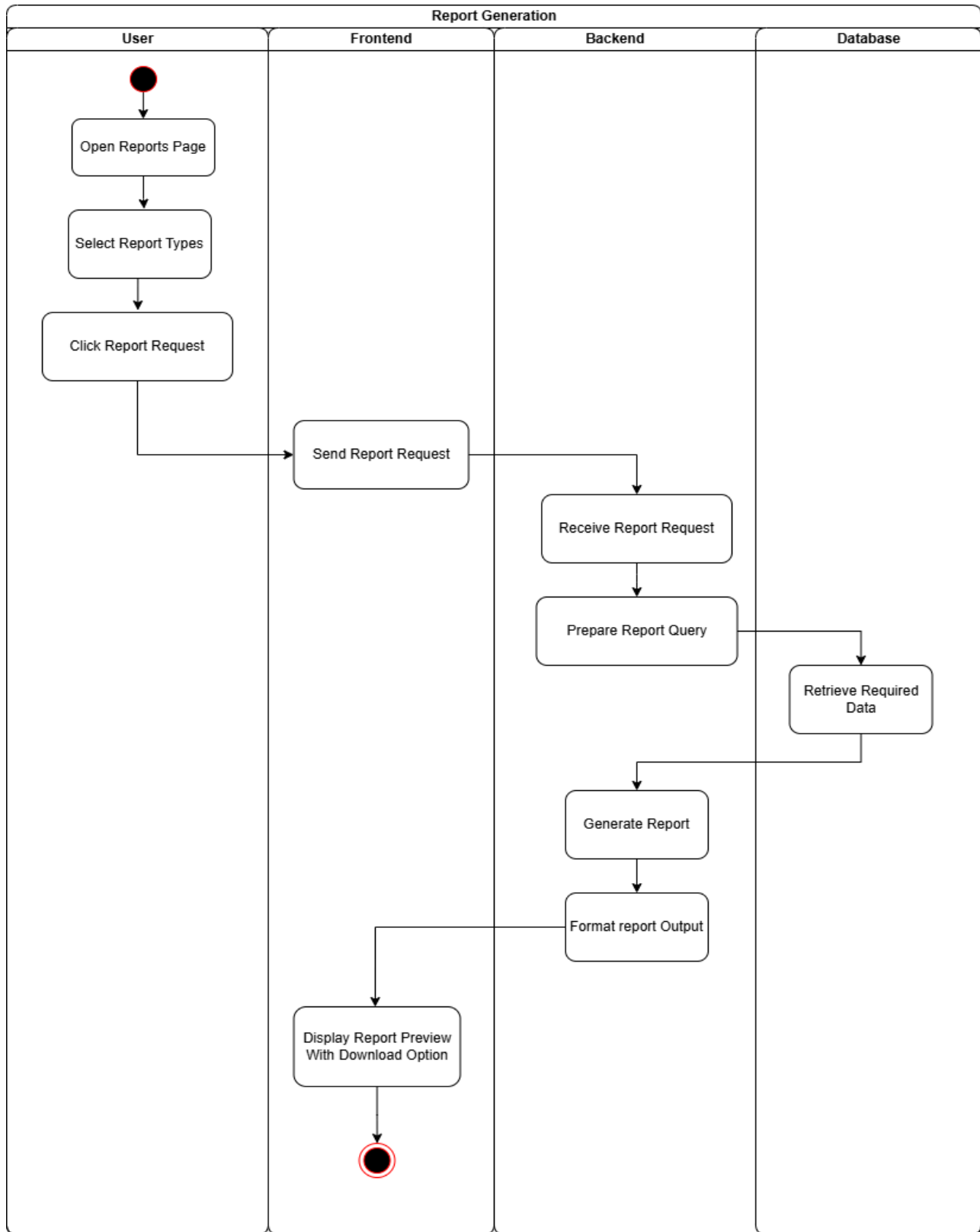


Figure 4.3.2.13 Warehouse Report Generation Activity Diagram

4.3.2.14 AI Chatbot

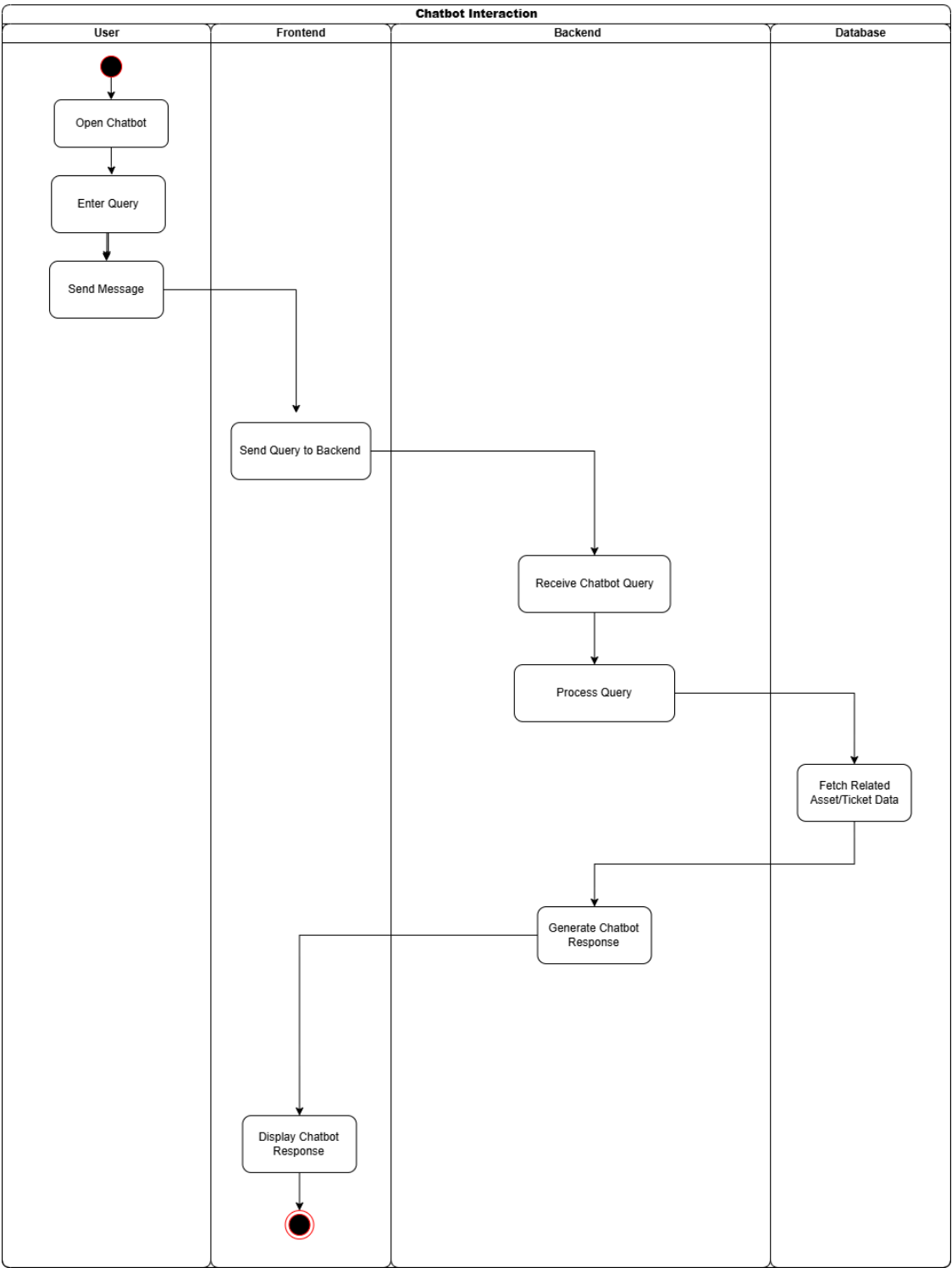


Figure 4.3.2.14 AI Chatbot Activity Diagram

4.3.4 Sequence Diagrams

4.3.4.1 Login UI

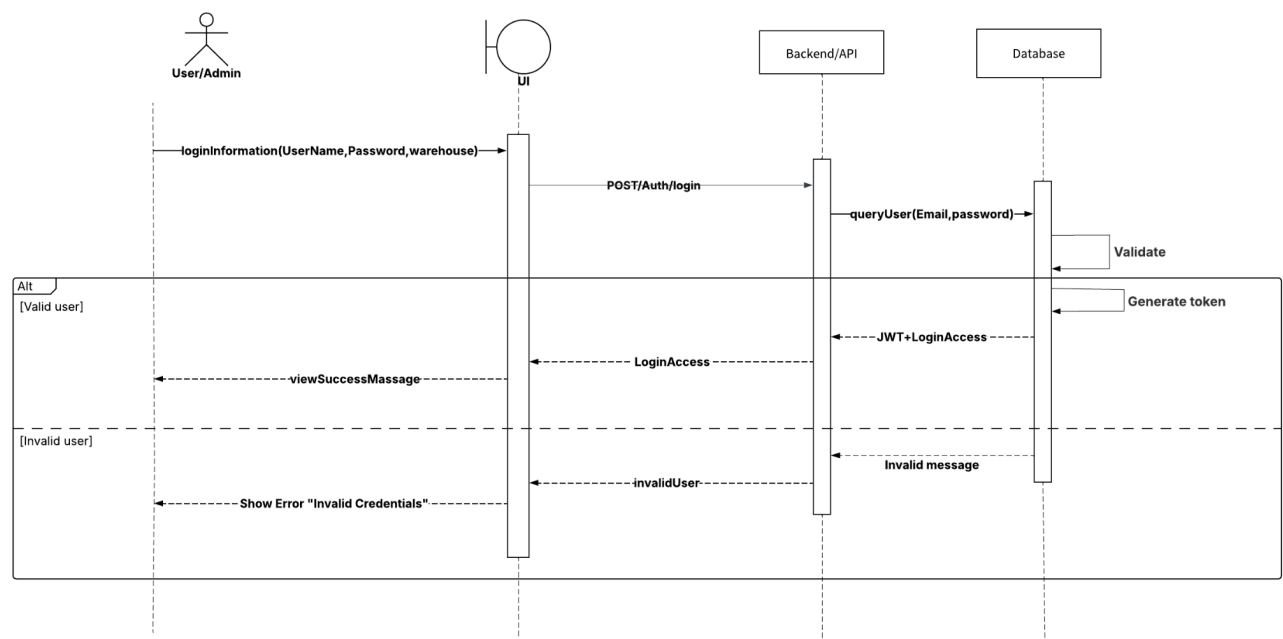


Figure 4.3.3.1 login UI Sequence Diagram

4.3.4.2 Asset Management (Create, View, Update, Delete Assets)

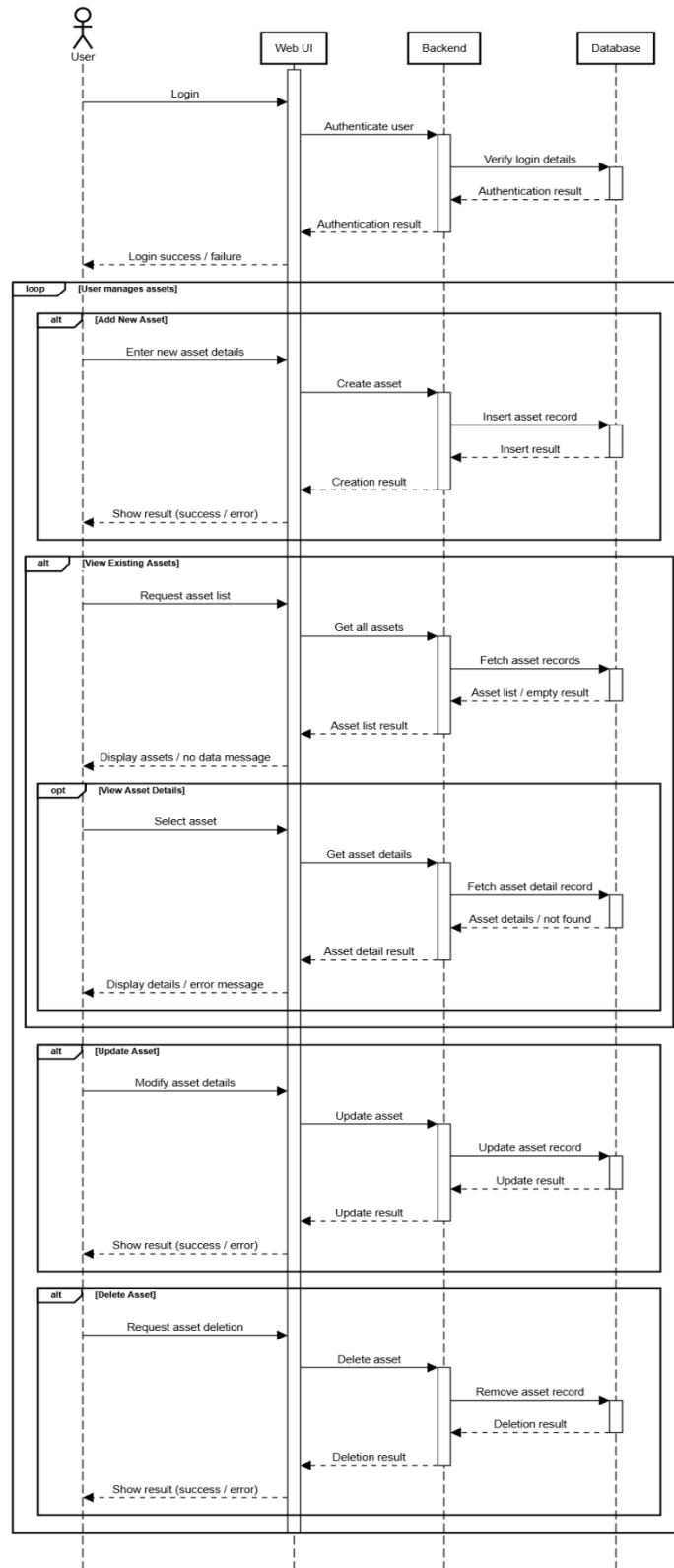


Figure 4.3.3.2 Asset Management (Create, View, Update, Delete Assets) Sequence Diagram

4.3.4.3 Search Assets

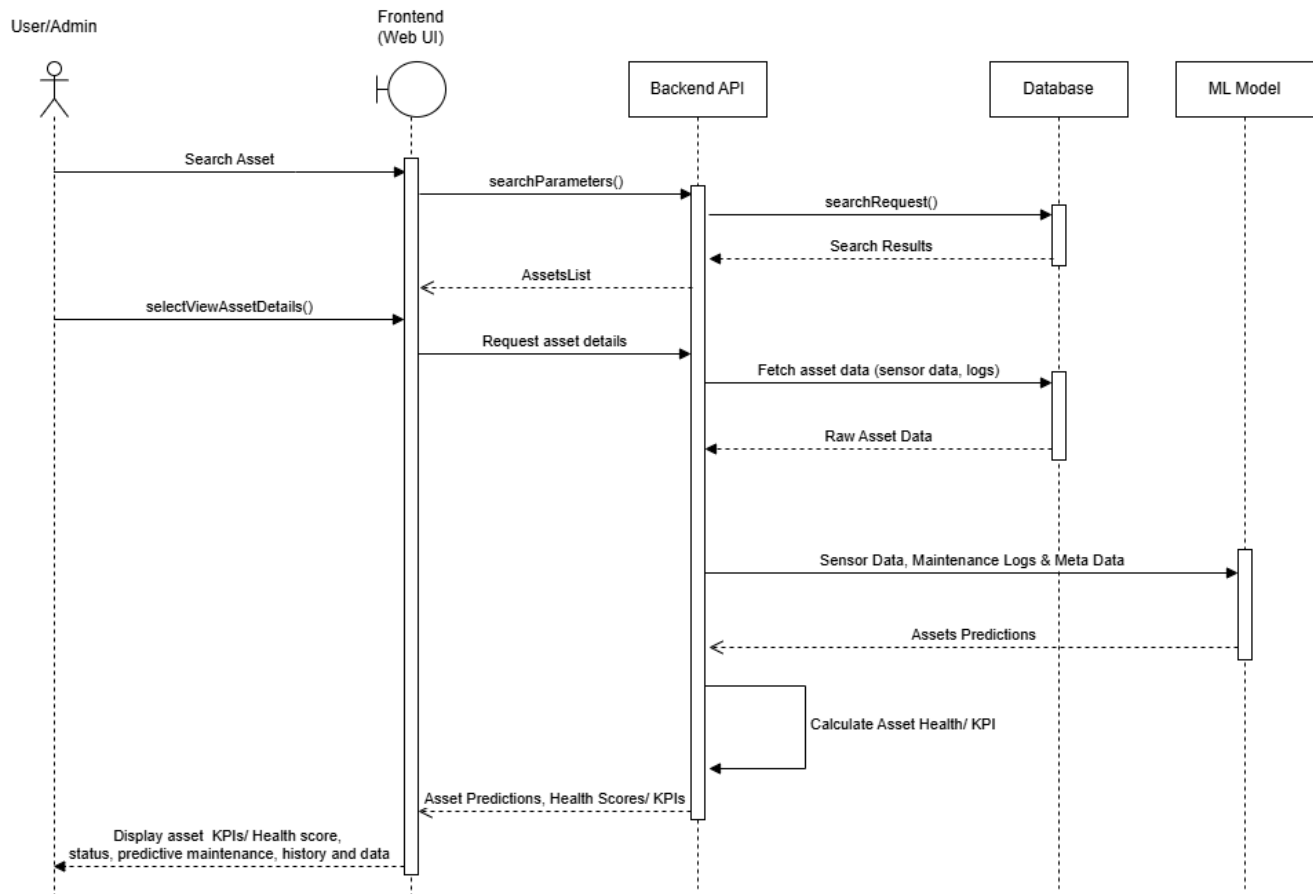


Figure 4.3.3.3 Search Assets Sequence Diagram

4.3.4.4 Asset Report Generation

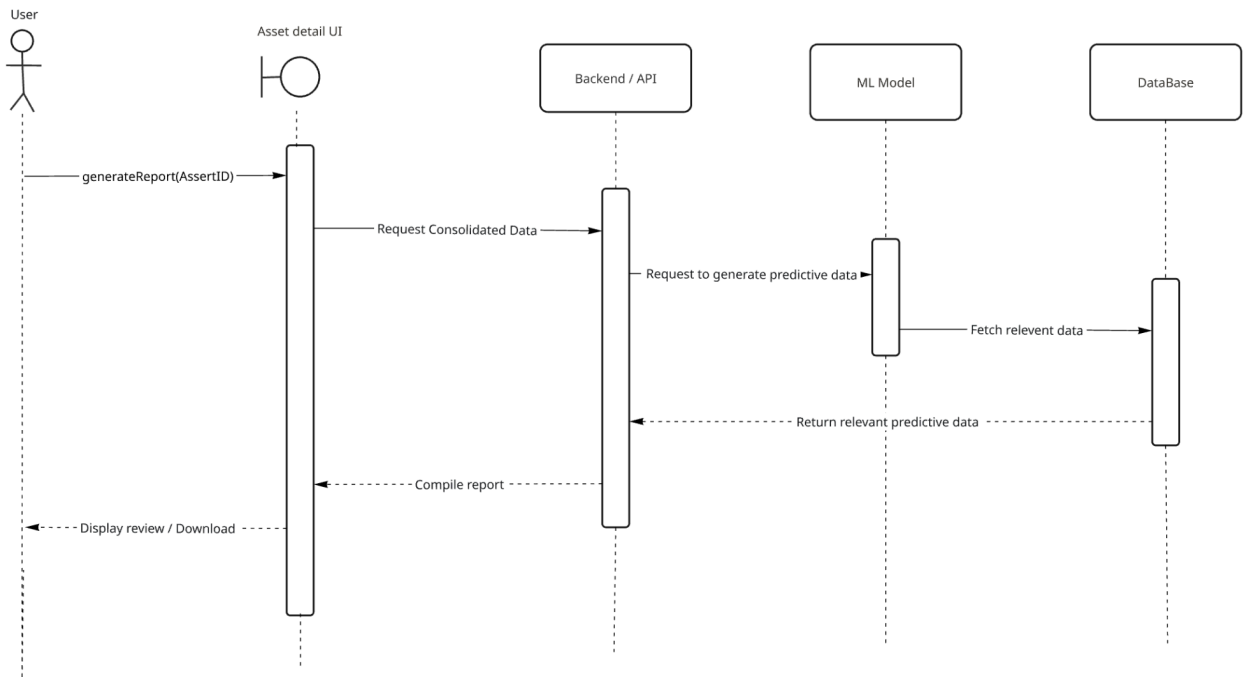


Figure 4.3.3.4 Asset Report Generation Sequence Diagram

4.3.4.5 Create Ticket

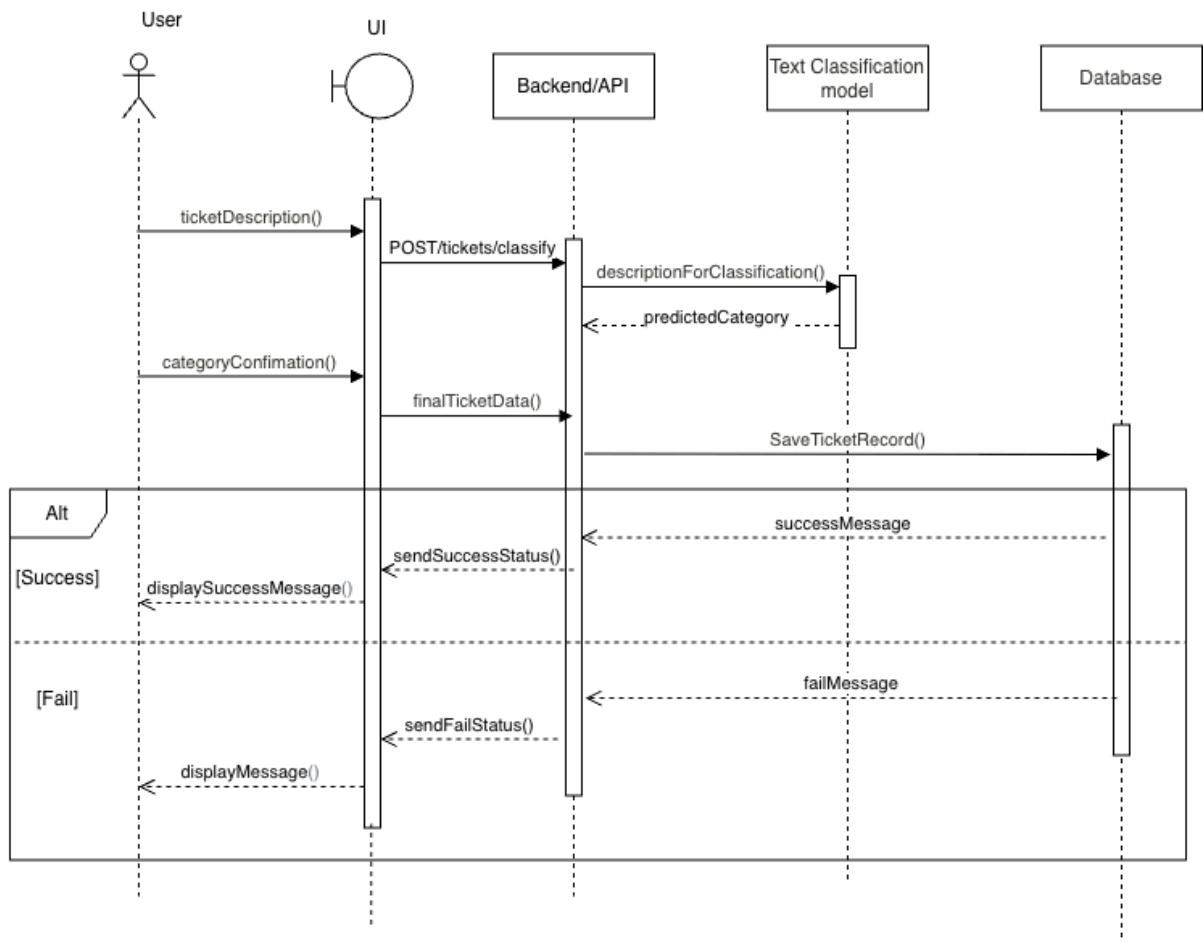


Figure 4.3.3.5 Create Ticket Sequence Diagram

4.3.4.6 View, Update, Delete tickets

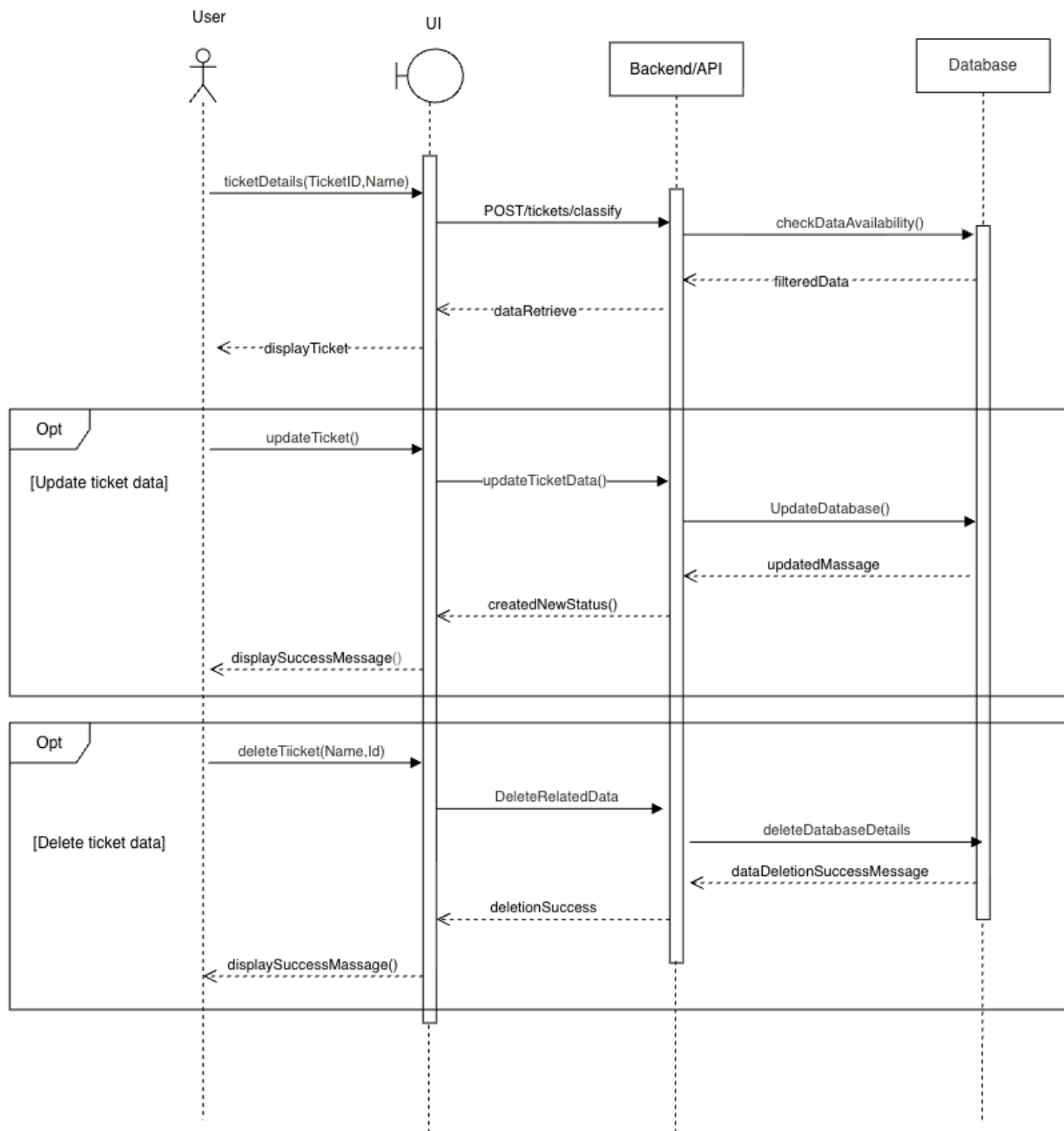


Figure 4.3.3. 6: View, Update, Delete Tickets Sequence Diagram

4.3.4.7 Review Ticket

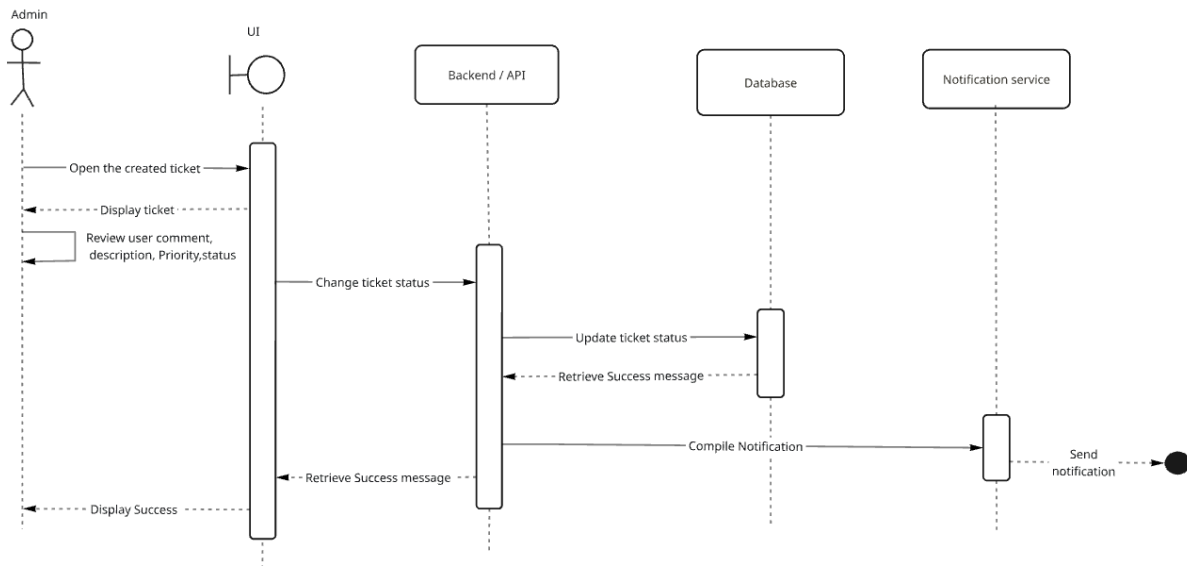


Figure 4.3.3.7: Review Ticket Sequence Diagram

4.3.4.8 View Warehouse Insights

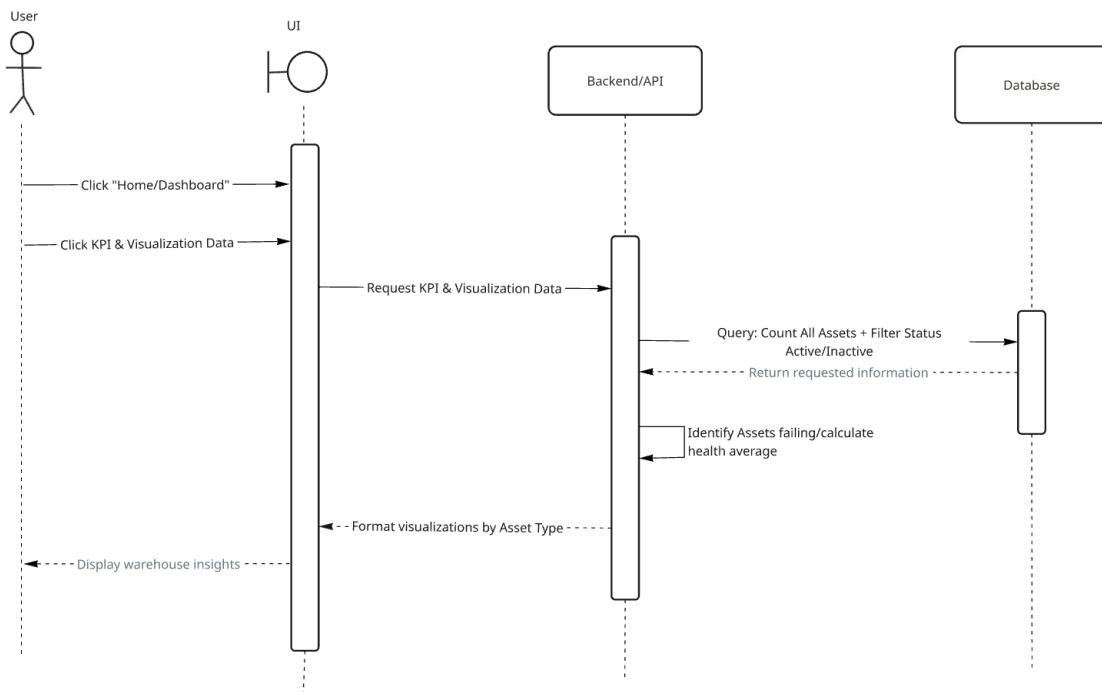


Figure 4.3.3.8 View Warehouse Insights Sequence Diagram

4.3.4.9 Warehouse Report Generation

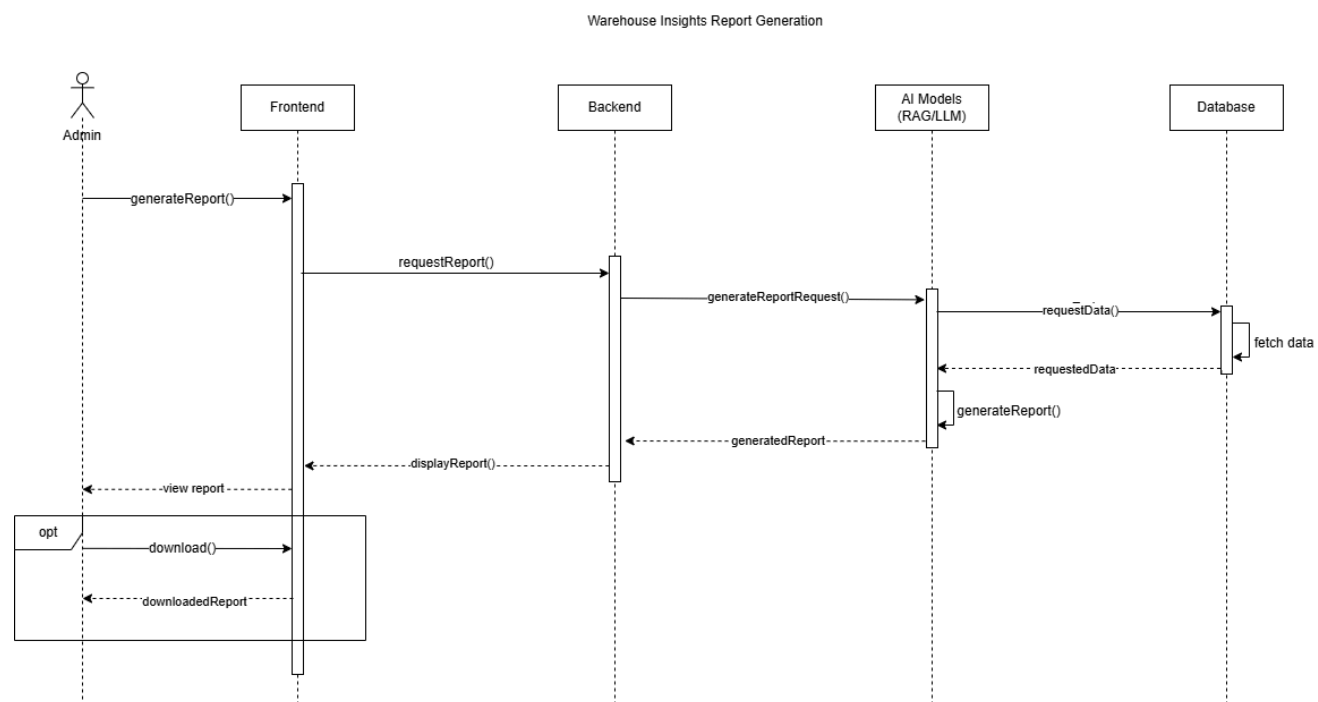


Figure 4.3.3.9 Warehouse Report Generation Sequence Diagram

4.3.4.10 Profile Profile

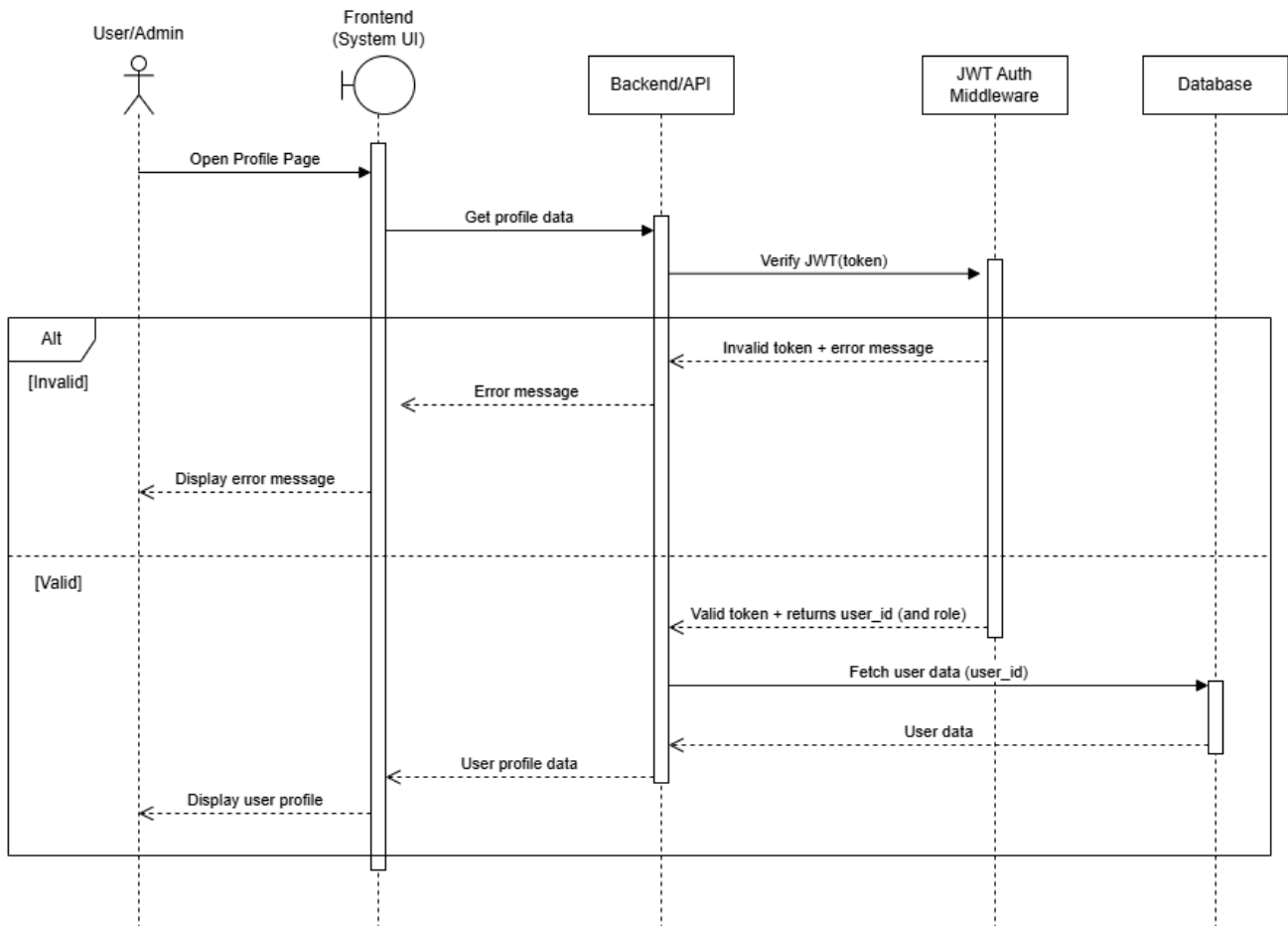


Figure 4.3.3.10 Profile Profile Sequence Diagram

4.3.4.11 User Management (Create, View, Update, Delete User Accounts)

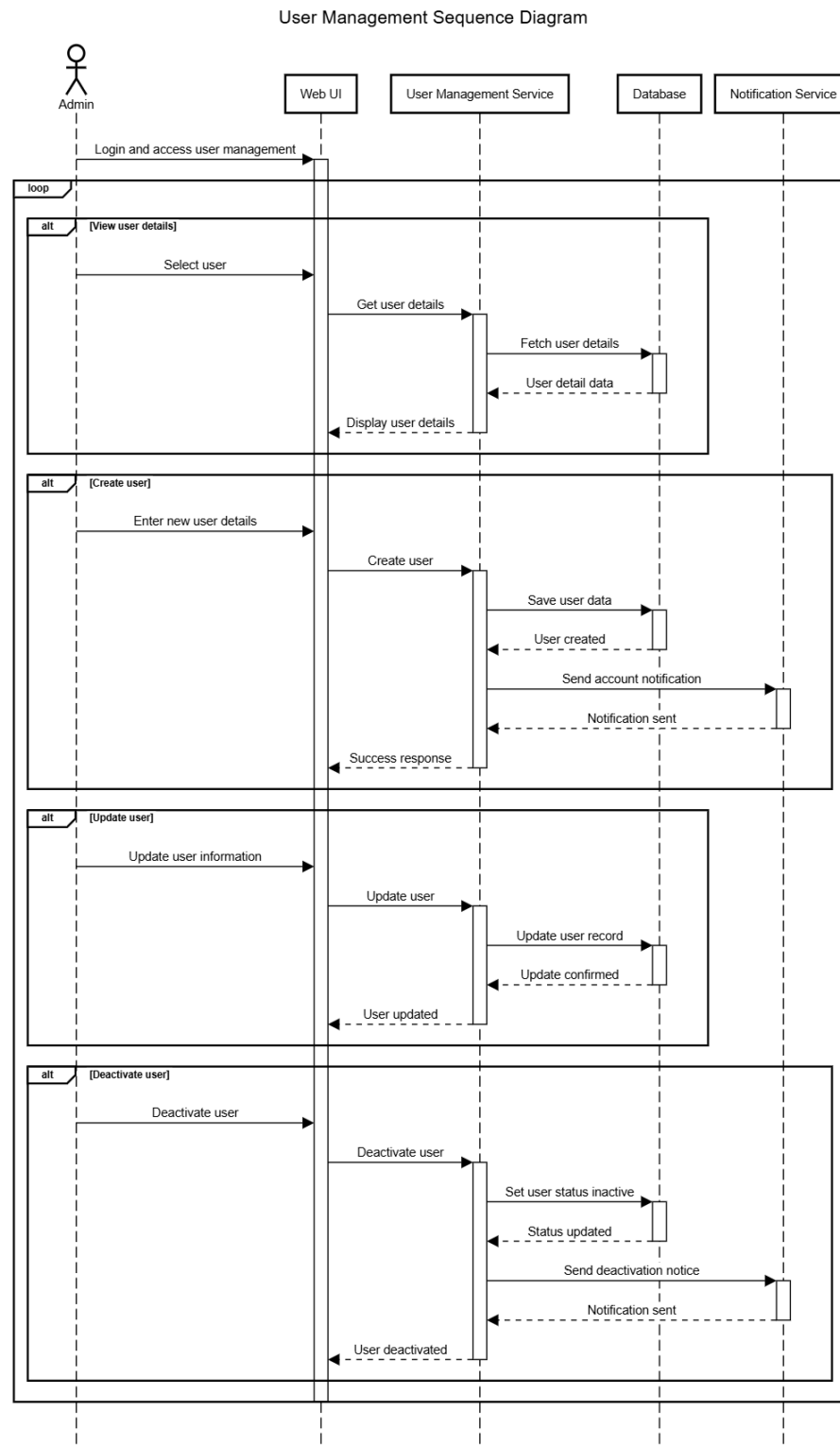


Figure 4.3.3.11 User Management (Create, View, Update, Delete User Accounts) Sequence Diagram

4.3.4.12 Search Users

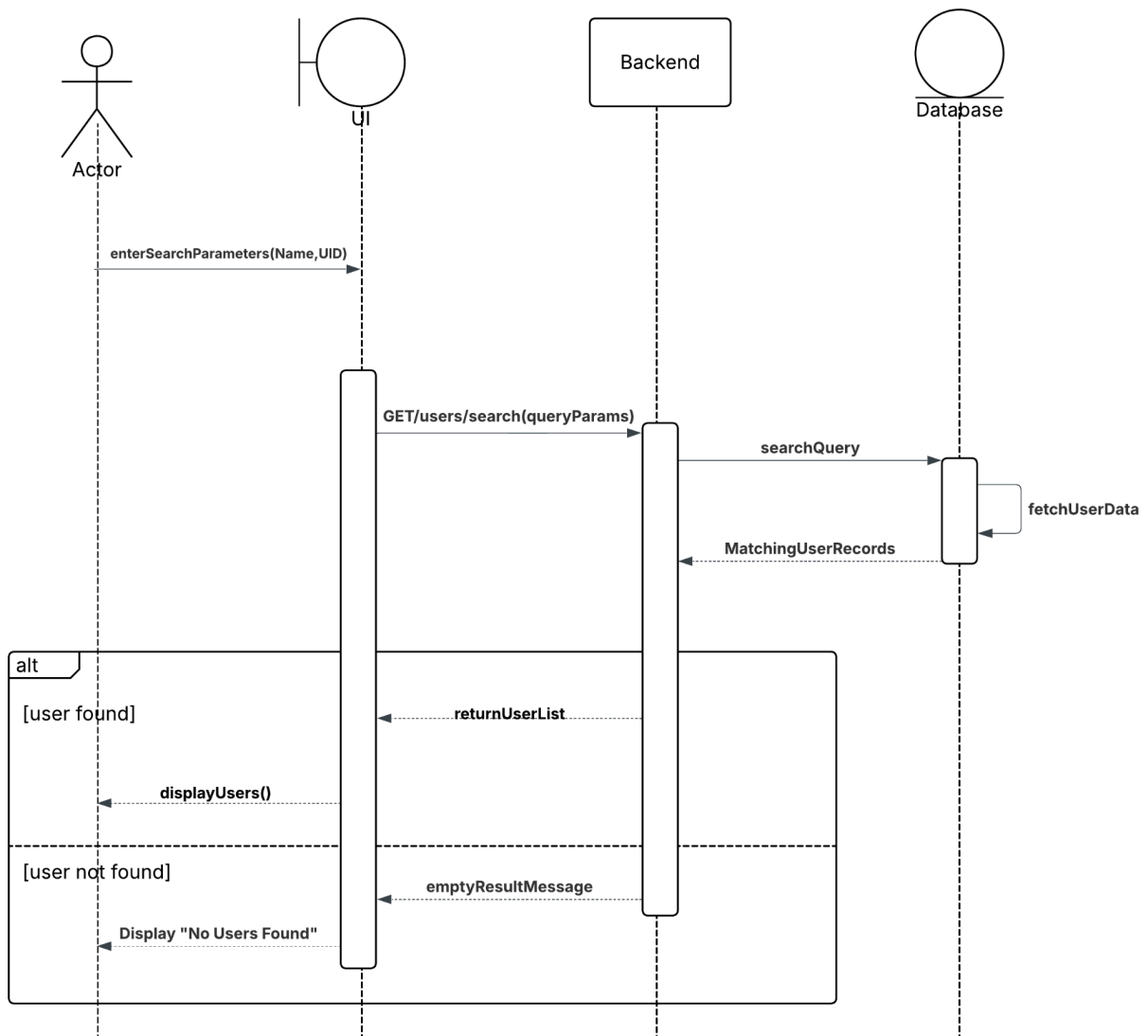


Figure 4.3.3.12 Search Users Sequence Diagram

4.3.3.13 AI Chatbot

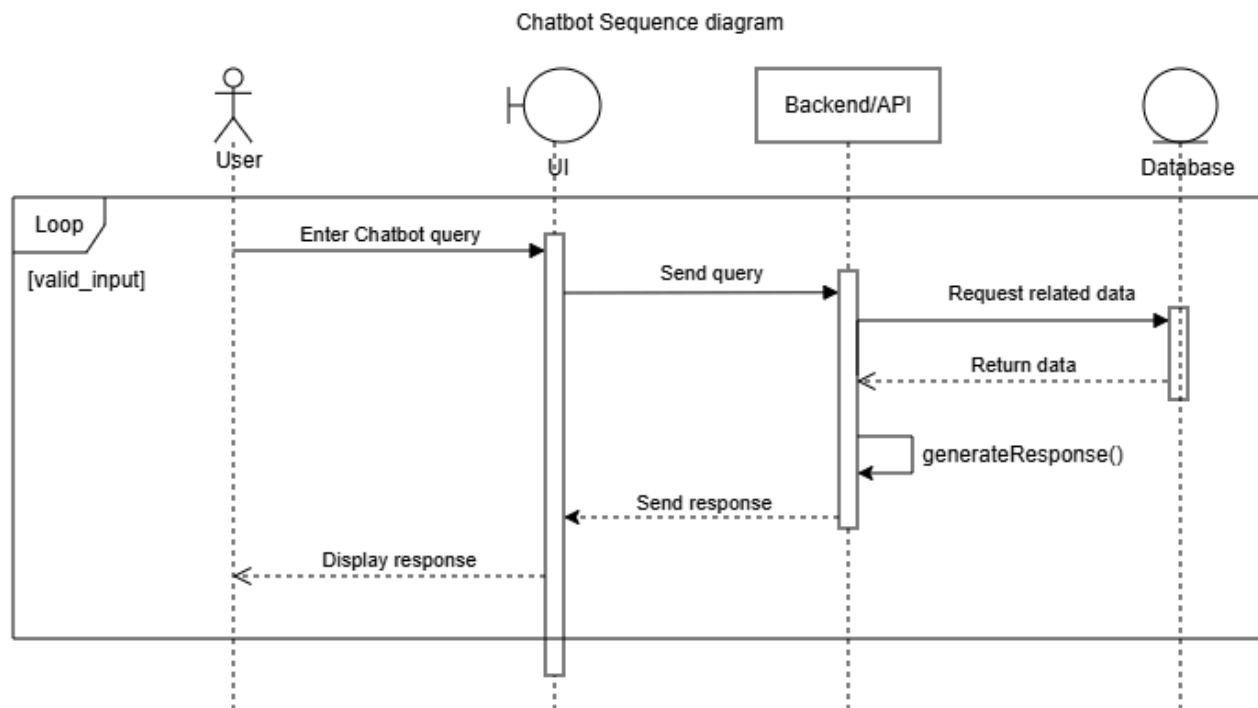


Figure 4.3.3.13 AI Chatbot Sequence Diagram

4.3.5 ER Diagram

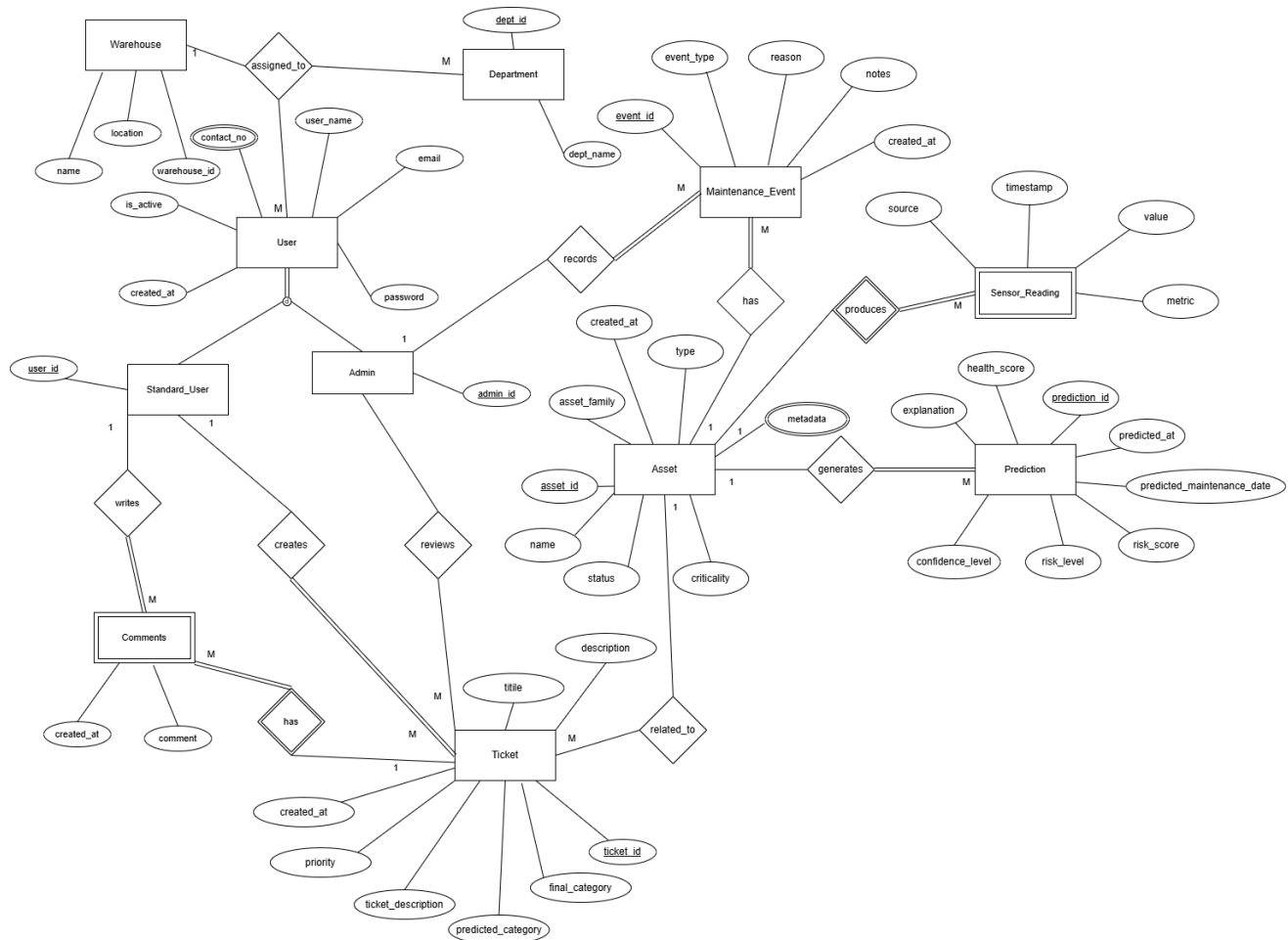


Figure 4.3.5 ER Diagram.

4.3.6 Class Diagram

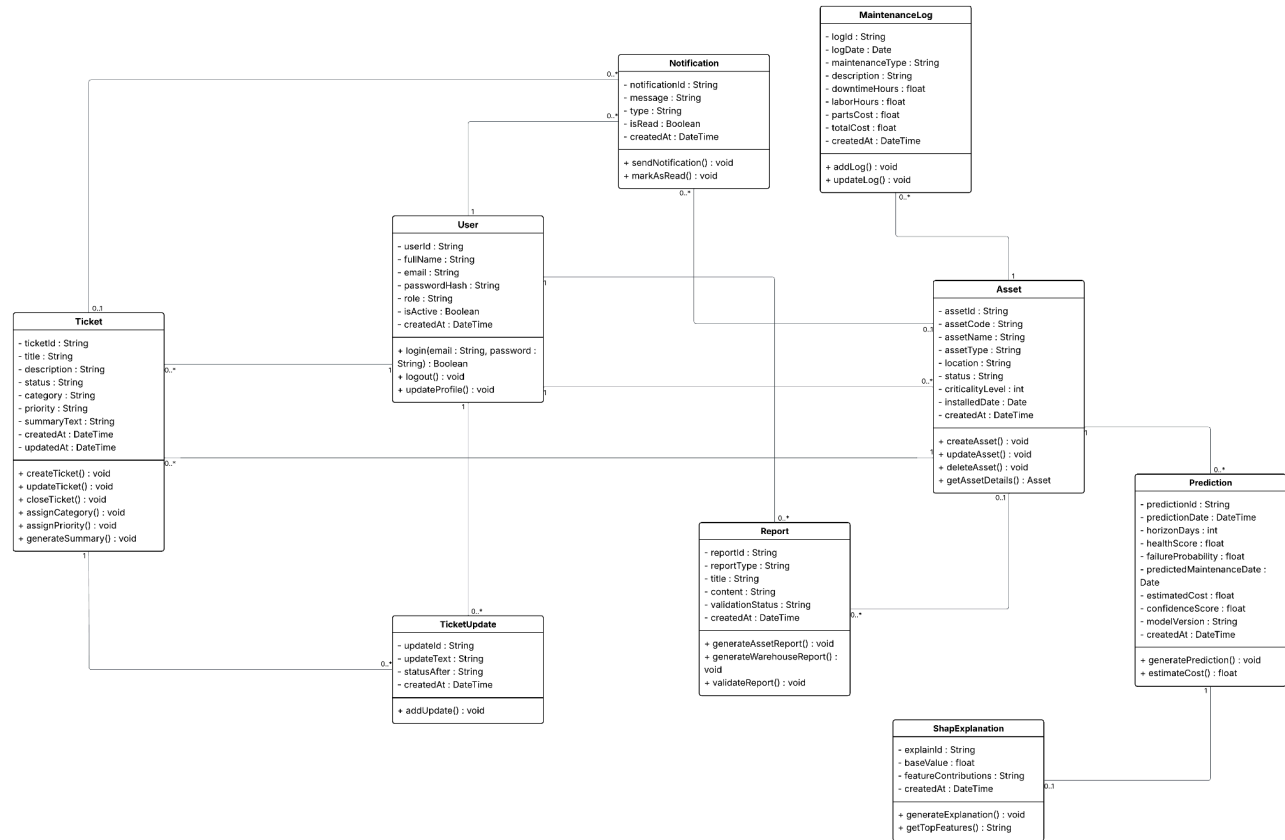


Figure 4.3.6 Class Diagram

4.3.7 Component Diagram

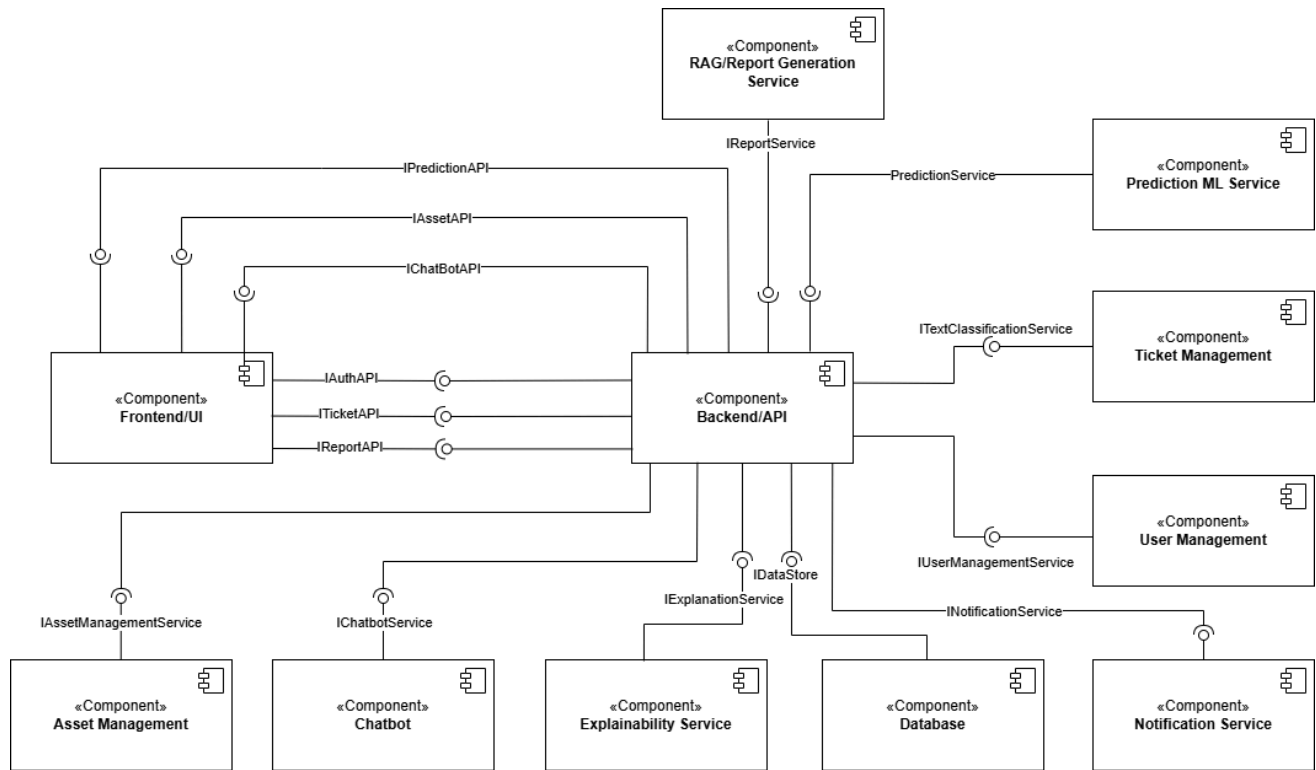


Figure 4.3.7 Component Diagram

Chapter 5: Implementation

5.1 Introduction

The implementation phase is one of the most important parts of this project, as it converts the system design into a working real-world application. The main focus of this phase was on developing the software components, configuring the system, and testing each feature to ensure that it functions correctly. Continuous testing was carried out throughout development to identify issues early and improve overall system performance.

This chapter describes how the proposed system was implemented in practice and explains the technologies used, the development methods followed, and the challenges faced during the process. The solutions adopted to overcome these challenges are also discussed. The main objective of this phase was to build a stable, user-friendly system that satisfies the defined project requirements.

5.2 Implementation Process

The implementation process began with collecting and refining system requirements through discussions with supervisors and by reviewing similar existing systems. Existing applications were studied to understand common workflows, core features, and user expectations. This approach helped the team gain a clearer understanding of system behavior and supported informed design and implementation decisions.

After completing the planning stage, development tasks were divided among team members based on their skills and responsibilities. This approach improved teamwork, balanced the workload, and supported skill development throughout the project.

The frontend of the application was developed using Next.js, which supports responsive, interactive, and visually consistent user interfaces. The component-based structure of Next.js made it easier to create reusable UI elements such as dashboards, tables, and forms, while maintaining consistency across different pages of the application.

The backend was implemented using FastAPI, a Python-based framework chosen for its high performance and support for asynchronous operations. FastAPI enabled efficient handling of API requests and simplified the integration of system logic with machine learning models and AI services. Real-time features such as dashboard updates and chatbot interactions were managed effectively using this framework, improving overall system responsiveness.

Python was used as the primary programming language for backend development due to its simplicity and strong support for web development and AI-related libraries. Python also helped speed up development, testing, and debugging of predictive maintenance models, ticket analysis components, and report generation features.

Supabase was used as the relational database management system. It was selected for its reliability, scalability, and ability to handle structured data efficiently. Supabase stores user information, asset records, maintenance logs, tickets, and predictive outputs securely. It also supports role-based access

control and integration with vector embeddings used in the retrieval-augmented generation process.

Artificial intelligence components, including predictive maintenance models, maintenance cost estimation, ticket categorization, summarization, and chatbot-based querying, were integrated into the backend during this phase. These components were tested iteratively to ensure accurate predictions, reliable explanations, and smooth interaction with dashboards and reports.

The system was designed to run on any device with an HTML5 compatible web browser and an active internet connection. Special attention was given to responsive design so that users can access the application comfortably from desktops, laptops, tablets, and smartphones.

5.3 Summary

The implementation phase was an important part of the PredictiX project, as it focused on turning the proposed system design into working software components. During this phase, several core features were implemented, initial prototypes were developed, and the main technologies were integrated to test the feasibility of the system. While not all planned functionalities were fully completed, steady progress was made in building the system foundation and validating the overall architecture.

The system was developed using Next.js for the frontend, FastAPI and Python for backend processing, and Supabase as the database solution. This technology stack supported the development of a modular and scalable application structure. Key components such as asset management interfaces, dashboard views, predictive maintenance models, ticket classification and summarization features, vector-based retrieval for RAG, and chatbot-related functionality were implemented or partially implemented during this stage.

AI-related components, including predictive maintenance using XGBoost, explainable AI techniques with SHAP, NLP-based ticket categorization and summarization, and retrieval-augmented report generation, were developed incrementally and tested alongside system features. Continuous testing helped identify issues related to data flow, performance, and integration between components. Some advanced features remain under further development; however, the current implementation demonstrates the practical applicability of the proposed approach.

At the end of the implementation phase, the PredictiX system provides a functional base capable of delivering initial, data-driven maintenance insights. The completed work establishes a solid foundation for future improvements, feature completion, and system expansion.

Chapter 6: References

- [1] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Oxford, UK: Butterworth-Heinemann, 2002.
- [2] A. K. S. Jardine, D. Lin, and D. Banjevic, “A review on machinery diagnostics and prognostics implementing condition-based maintenance,” *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] IBM, *IBM Maximo Application Suite Documentation*, IBM Corporation, 2023.
- [5] React, “React Documentation,” 2024. Available: <https://react.dev>
- [7] Supabase, “Supabase Documentation,” 2024. Available: <https://supabase.com/docs>
- [8] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2024. Available: <https://www.postgresql.org/docs/>
- [9] J. Brownlee, *Machine Learning Mastery with Python*. Machine Learning Mastery, 2020.
- [10] ISO, *ISO 55000: Asset Management – Overview, Principles and Terminology*, International Organization for Standardization, 2018.
- [11] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.
- [12] Carvalho, T. P., et al. (2019 - *A systematic literature review of machine learning methods applied to predictive maintenance*. Computers & Industrial Engineering, 137, 106024.
- [13] Chen, T., & Guestrin, C. (2016) - *XGBoost: A scalable tree boosting system*, Proceedings of the 22nd ACM SIGKDD Conference.
- [14] Lundberg, S. M., & Lee, S.-I. (2017) - *A unified approach to interpreting model predictions*, Advances in Neural Information Processing Systems (NeurIPS).
- [15] Arrieta, A. B., et al. (2020) - *Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges*. Information Fusion, 58, 82–115.
- [16] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019) - *BERT: Pre-training of deep bidirectional transformers for language understanding*. NAACL-HLT.
- [17] Sanh, V., et al. (2019) - *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108.
- [18] Lewis, M., et al. (2020) - *BART: Denoising sequence-to-sequence pre-training for natural language generation*. ACL 2020.
- [19] pgvector: Open-source vector similarity search for Postgres. - <https://github.com/pgvector/pgvector>

- [20] Ramírez, S. (FastAPI Author). - *FastAPI Documentation*. - <https://fastapi.tiangolo.com/>
- [21] Jones, M., Bradley, J., & Sakimura, N. (2015). - *JSON Web Token (JWT)* - IETF RFC 7519.
- [22] Asset Panda - <https://www.assetpanda.com/>
- [23] ManageEngine AssetExplorer - <https://www.manageengine.com/products/asset-explorer/>
- [24] Reftab - <https://www.reftab.com/>
- [25] IBM Maximo - <https://www.ibm.com/products/maximo/asset-management>
- [26] InvGate - <https://invgate.com/asset-management>
- [27] Wasp AssetCloud Complete - <https://www.waspbarcode.com/asset-tracking/cloud>
- [28] Lansweeper - <https://www.lansweeper.com/>
- [29] Flexera One - <https://www.flexera.com/products/flexera-one>

Chapter 7: Appendix- Individual Contributions

Index Number	Name	Contributions	To Do
235514B	Ekanayake D.B	- Conducted literature review on existing systems: - Asset Panda - ManageEngine - Researched warehouse and asset failure prediction models and explainable AI (XAI) techniques (SHAP) - Developed concepts for report generation and multi-step validation using RAG and LLMs - Developed the initial prototype of the system - Configured Supabase for the project - Create Github repositories for frontend and backend and develop the project's initial implementations. - Developed the ticket category classification model by fine-tuning DistilBERT base uncased model - Developments in UI - Login Page - Initial Dashboard - Designed UML diagrams: - Use Case Diagram - ER Diagram - Class Diagram - Component diagram - Sequence diagrams: - Search assets	- AI Components - Develop predictive maintenance model for assets using XGBoost - Develop Explainable AI (XAI) - SHAP for predictions - Develop vector database using sentence transformer for retrieval augment generation (RAG) - Frontend - Develop assets section - Develop dashboard section - Backend - Develop assets section - Develop dashboard section

		■ View Profile ■ Asset Management ■ AI Chatbot ○ Activity diagrams: ■ View, Update, Delete tickets ■ Asset report generation ■ Create ticket ■ Search assets ● Contributed to make the Software Requirements Specification (SRS) and Interim Report	
235502L	Abeywickrama W.S.J.	● Conducted literature review on existing systems: ○ Flexera One ○ Reftab ● Researched NLP based ticket summarization techniques for overall and individual ticket summaries ● Designed UML diagrams: ○ Sequence diagrams: ■ Warehouse report generation ○ Activity diagrams: ■ Report generation ■ AI chatbot ■ View warehouse insight ■ Login ■ Warehouse report generation ● Contributed to make the Software Requirements Specification (SRS) and Interim Report ● Contributed to make presentation	● AI Components ○ Develop NLP based (BART) ticket summary generation model ○ Develop NLP based summary generation model for relevant ticket's asset's current data ○ Develop warehouse insight report generation using RAG and LLMs ● Frontend ○ Develop warehouse insights section ○ Develop dashboard section ● Backend ○ Develop warehouse insights section ○ Asset notification service

235536T	Senevirathna P.T.B	- Conducted literature review on existing systems: - IBM Maximo - InvGate - Researched asset level report generation using RAG and LLMs for future implementation - Designed UML diagrams: - Sequence diagrams: - Login - Review ticket - View warehouse insights - Asset report generation - Predictive maintenance asset - Activity diagrams: - Search ticket - Review tickets - Search user - Contributed to make the Software Requirements Specification (SRS) and Interim Report	- AI Components - Develop asset report generation using RAG and LLMs - Cost estimation model using XGBoost - Frontend - Develop users section with authentication - Develop profile page - Backend - Develop users section - Develop profile page - Critical asset prediction notification service
235547D	Wijesinghe W.M.A.N	- Conducted literature review on existing systems: - Wasp AssetCloud - Lansweeper - Researched chatbot implementations using LLMs and RAG for warehouse related use cases - Designed UML diagrams: - Sequence diagrams : - View, Update, Delete user - Search users - Create ticket - View, Update Delete ticket	- AI Components - Develop RAG and LLM based AI Chatbot - Develop NLP based (BERT) ticket priority classification model - Frontend - Develop tickets Section - Develop ticket notification service - Develop about page - Backend - Develop ticket section

		○ Activity diagrams : ■ View, Update, Delete user ■ View profile ● Contributed to make the Software Requirements Specification (SRS) and Interim Report	
--	--	--	--